Metodología para la Resolución de Problemas de Programación

Plantilla de análisis, diseño, codificación y pruebas

112 Ejercicios de Cadenas en Java (Tarea 11: Tarea11_001 a Tarea11_112)

Ejercicio 1: Tarea11_001 - Obtener Carácter en un Índice Específico

Comprensión del Problema	
Descripción del problema:	Obtener Carácter en un Índice Específico. Write a Java program to get the character at the given index within the string.
Entradas requeridas:	String str = "Java Exercises!" int index1 = 0 int index2 = 10
Salidas esperadas:	Original String = Java Exercises! The character at position 0 is J The character at position 10 is i
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para obtener carácter en un índice específico utilizando operaciones de la clase String (String.charAt()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"Java Exercises!"' int index1: Valor numérico o índice (0) int index2: Valor numérico o índice (10)
Operaciones , Formulas y Reglas :	Métodos aplicados: String.charAt(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	Original String = Java Exercises! The character at position 0 is J The character at position 10 is i
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.

¿Qué operaciones serán necesarias?	Invocación de métodos String.charAt() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 2: Tarea11_002 - Obtener Punto de Código Unicode en un Índice

Comprensión del Problema	
Descripción del problema:	Obtener Punto de Código Unicode en un Índice. Write a Java program to get the character (Unicode code point) at the given index within the string.
Entradas requeridas:	String str = "w3resource.com" int val1 = str.codePointAt(1) int val2 = str.codePointAt(10)
Salidas esperadas:	Original String : w3resource.com Character(unicode point) = 51 Character(unicode point) = 101
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para obtener punto de código unicode en un índice utilizando operaciones de la clase String (String.codePointAt()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"w3resource.com"' int val1: Valor numérico o índice (str.codePointAt(1)) int val2: Valor numérico o índice (str.codePointAt(10))
Operaciones , Formulas y Reglas :	Métodos aplicados: String.codePointAt(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	Original String : w3resource.com Character(unicode point) = 51 Character(unicode point) = 101
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos.

	2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.codePointAt() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 3: Tarea11_003 - Obtener Punto de Código Unicode Previo a un Índice

Comprensión del Problema	
Descripción del problema:	Obtener Punto de Código Unicode Previo a un Índice. Write a Java program to get the character (Unicode code point) before the specified index within the string.
Entradas requeridas:	String str = "w3resource.com" int val1 = str.codePointBefore(1) int val2 = str.codePointBefore(9)
Salidas esperadas:	Original String : w3resource.com Character(unicode point) = 119 Character(unicode point) = 99
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para obtener punto de código unicode previo a un índice utilizando operaciones de la clase String (String.codePointBefore()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"w3resource.com"' int val1: Valor numérico o índice (str.codePointBefore(1)) int val2: Valor numérico o índice (str.codePointBefore(9))
Operaciones , Formulas y Reglas :	Métodos aplicados: String.codePointBefore(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	Original String : w3resource.com Character(unicode point) = 119 Character(unicode point) = 99

Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.codePointBefore() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 4: Tarea11_004 - Contar Puntos de Código Unicode en un Rango

Comprensión del Problema	
Descripción del problema:	Contar Puntos de Código Unicode en un Rango. Write a Java program to count Unicode code points in the specified text range of a string.
Entradas requeridas:	String str = "w3rsource.com" int count = str.codePointCount(1, 10) int count = " + count)
Salidas esperadas:	Original String : w3rsource.com Codepoint count = 9
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para contar puntos de código unicode en un rango utilizando operaciones de la clase String (String.codePointCount()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"w3rsource.com"' int count: Valor numérico o índice (str.codePointCount(1, 10)) int count: Valor numérico o índice (" + count))
Operaciones , Formulas y Reglas :	Métodos aplicados: String.codePointCount(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	Original String : w3rsource.com

	Codepoint count = 9
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.codePointCount() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 5: Tarea11_005 - Comparación Lexicográfica entre Dos Cadenas

Comprensión del Problema	
Descripción del problema:	Comparación Lexicográfica entre Dos Cadenas. Write a Java program to compare two strings lexicographically. Two strings are lexicographically equal if they are the same length and contain the same characters in the same positions.
Entradas requeridas:	String str1 = "This is Exercise 1" String str2 = "This is Exercise 2" int result = str1.compareTo(str2)
Salidas esperadas:	String 1: This is Exercise 1 String 2: This is Exercise 2 "This is Exercise 1" is less than "This is Exercise 2"
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para comparación lexicográfica entre dos cadenas utilizando operaciones de la clase String (String.compareTo()).
Variables identificadas(tipo y uso)	String str1: Almacena la cadena de texto '"This is Exercise 1"' String str2: Almacena la cadena de texto '"This is Exercise 2"'

	int result: Valor numérico o índice (str1.compareTo(str2))
Operaciones , Formulas y Reglas :	Métodos aplicados: String.compareTo(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	String 1: This is Exercise 1 String 2: This is Exercise 2 "This is Exercise 1" is less than "This is Exercise 2"
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.compareTo() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 6: Tarea11_006 - Comparación Lexicográfica Ignorando Mayúsculas y Minúsculas

Comprensión del Problema	
Descripción del problema:	Comparación Lexicográfica Ignorando Mayúsculas y Minúsculas. Write a Java program to compare two strings lexicographically, ignoring case differences.
Entradas requeridas:	String str1 = "This is exercise 1" String str2 = "This is Exercise 1" int result = str1.compareToIgnoreCase(str2)
Salidas esperadas:	String 1: This is exercise 1 String 2: This is Exercise 1 "This is exercise 1" is equal to "This is Exercise 1"
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para comparación lexicográfica ignorando

	mayúsculas y minúsculas utilizando operaciones de la clase String (String.compareToIgnoreCase()).
Variables identificadas(tipo y uso)	String str1: Almacena la cadena de texto '"This is exercise 1"' String str2: Almacena la cadena de texto '"This is Exercise 1"' int result: Valor numérico o índice (str1.compareToIgnoreCase(str2))
Operaciones , Formulas y Reglas :	Métodos aplicados: String.compareToIgnoreCase(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	String 1: This is exercise 1 String 2: This is Exercise 1 "This is exercise 1" is equal to "This is Exercise 1"
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.compareToIgnoreCase() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 7: Tarea11_007 - Concatenación de Dos Cadenas de Texto

Comprensión del Problema	
Descripción del problema:	Concatenación de Dos Cadenas de Texto. Write a Java program to concatenate a given string to the end of another string.
Entradas requeridas:	String str1 = "PHP Exercises and " String str2 = "Python Exercises"
Salidas esperadas:	String 1: PHP Exercises and String 2: Python Exercises The concatenated string: PHP Exercises and Python Exercises
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.

Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para concatenación de dos cadenas de texto utilizando operaciones de la clase String (String.concat()).
Variables identificadas(tipo y uso)	String str1: Almacena la cadena de texto '"PHP Exercises and "' String str2: Almacena la cadena de texto '"Python Exercises"'
Operaciones , Formulas y Reglas :	Métodos aplicados: String.concat(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	String 1: PHP Exercises and String 2: Python Exercises The concatenated string: PHP Exercises and Python Exercises
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.concat() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 8: Tarea11_008 - Verificar si una Cadena Contiene una Secuencia de Caracteres

Comprensión del Problema	
Descripción del problema:	Verificar si una Cadena Contiene una Secuencia de Caracteres. Write a Java program to test if a given string contains the specified sequence of char values.
Entradas requeridas:	String str = "PHP Exercises and Python Exercises" String seq = "and"
Salidas esperadas:	Original String: PHP Exercises and Python Exercises Specified sequence of char values: and true

Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para verificar si una cadena contiene una secuencia de caracteres utilizando operaciones de la clase String (String.contains()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"PHP Exercises and Python Exercises"' String seq: Almacena la cadena de texto '"and"'
Operaciones , Formulas y Reglas :	Métodos aplicados: String.contains(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	Original String: PHP Exercises and Python Exercises Specified sequence of char values: and true
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.contains() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 9: Tarea11_009 - Comparar Cadena con una Secuencia de Caracteres (CharSequence)

Comprensión del Problema	
Descripción del problema:	Comparar Cadena con una Secuencia de Caracteres (CharSequence). Write a Java program to compare a given string to the specified character sequence.
Entradas requeridas:	String str1 = "example.com"
Salidas esperadas:	Comparing example.com and example.com: true

	Comparing Example.com and example.com: false
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para comparar cadena con una secuencia de caracteres (charsequence) utilizando operaciones de la clase String (String.contentEquals()).
Variables identificadas(tipo y uso)	String str1: Almacena la cadena de texto '"example.com"'
Operaciones , Formulas y Reglas :	Métodos aplicados: String.contentEquals(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	Comparing example.com and example.com: true Comparing Example.com and example.com: false
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.contentEquals() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 10: Tarea11_010 - Comparar Cadena con un Búfer de Cadenas (StringBuffer)

Comprensión del Problema	
Descripción del problema:	Comparar Cadena con un Búfer de Cadenas (StringBuffer). Write a Java program to compare a given string to a specified string buffer.
Entradas requeridas:	String str1 = "example.com"
Salidas esperadas:	Comparing example.com and example.com: true Comparing Example.com and example.com: false

Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para comparar cadena con un búfer de cadenas (stringbuffer) utilizando operaciones de la clase String (String.contentEquals()).
Variables identificadas(tipo y uso)	String str1: Almacena la cadena de texto '"example.com"'
Operaciones , Formulas y Reglas :	Métodos aplicados: String.contentEquals(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	Comparing example.com and example.com: true Comparing Example.com and example.com: false
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.contentEquals() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 11: Tarea11_011 - Crear un Objeto String a partir de un Arreglo de Caracteres

Comprensión del Problema	
Descripción del problema:	Crear un Objeto String a partir de un Arreglo de Caracteres. Write a Java program to create a String object with a character array.
Entradas requeridas:	Cadenas o parámetros de texto definidos en el método principal (main).
Salidas esperadas:	The book contains 234 pages.
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.

Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para crear un objeto string a partir de un arreglo de caracteres utilizando operaciones de la clase String (String.copyValueOf()).
Variables identificadas(tipo y uso)	Variables de tipo String, int y char según los argumentos requeridos por la lógica.
Operaciones , Formulas y Reglas :	Métodos aplicados: String.copyValueOf(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The book contains 234 pages.
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.copyValueOf() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 12: Tarea11_012 - Comprobar si una Cadena Termina con una Subcadena Específica

Comprensión del Problema	
Descripción del problema:	Comprobar si una Cadena Termina con una Subcadena Específica. Write a Java program to check whether a given string ends with another string.
Entradas requeridas:	String str1 = "Python Exercises" String str2 = "Python Exercise" String end_str = "se"
Salidas esperadas:	"Python Exercises" ends with "se"? false "Python Exercise" ends with "se"? true
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.

Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para comprobar si una cadena termina con una subcadena específica utilizando operaciones de la clase String (String.endsWith()).
Variables identificadas(tipo y uso)	String str1: Almacena la cadena de texto '"Python Exercises"' String str2: Almacena la cadena de texto '"Python Exercise"' String end_str: Almacena la cadena de texto '"se"'
Operaciones , Formulas y Reglas :	Métodos aplicados: String.endsWith(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	"Python Exercises" ends with "se"? false "Python Exercise" ends with "se"? true
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.endsWith() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 13: Tarea11_013 - Comprobar si Dos Cadenas Contienen los Mismos Datos (equals)

Comprensión del Problema	
Descripción del problema:	Comprobar si Dos Cadenas Contienen los Mismos Datos (equals). Write a Java program to check whether two String objects contain the same data.
Entradas requeridas:	String columnist1 = "Stephen Edwin King" String columnist2 = "Walter Winchell" String columnist3 = "Mike Royko"
Salidas esperadas:	"Stephen Edwin King" equals "Walter Winchell"? false "Stephen Edwin King" equals "Mike Royko"? false

Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para comprobar si dos cadenas contienen los mismos datos (equals) utilizando operaciones de la clase String (String.equals()).
Variables identificadas(tipo y uso)	String columnist1: Almacena la cadena de texto '"Stephen Edwin King"' String columnist2: Almacena la cadena de texto '"Walter Winchell"' String columnist3: Almacena la cadena de texto '"Mike Royko"'
Operaciones , Formulas y Reglas :	Métodos aplicados: String.equals(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	"Stephen Edwin King" equals "Walter Winchell"? false "Stephen Edwin King" equals "Mike Royko"? false
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.equals() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 14: Tarea11_014 - Comprobar si Dos Cadenas son Iguales Ignorando Mayúsculas y Minúsculas

Comprensión del Problema	
Descripción del problema:	Comprobar si Dos Cadenas son Iguales Ignorando Mayúsculas y Minúsculas. Write a Java program to compare a given string to another string, ignoring case considerations.
Entradas requeridas:	String columnist1 = "Stephen Edwin King"

	String columnist2 = "Walter Winchell" String columnist3 = "stephen edwin king"
Salidas esperadas:	"Stephen Edwin King" equals "Walter Winchell"? false "Stephen Edwin King" equals "stephen edwin king"? true
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para comprobar si dos cadenas son iguales ignorando mayúsculas y minúsculas utilizando operaciones de la clase String (String.equalsIgnoreCase()).
Variables identificadas(tipo y uso)	String columnist1: Almacena la cadena de texto '"Stephen Edwin King"' String columnist2: Almacena la cadena de texto '"Walter Winchell"' String columnist3: Almacena la cadena de texto '"stephen edwin king"'
Operaciones , Formulas y Reglas :	Métodos aplicados: String.equalsIgnoreCase(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	"Stephen Edwin King" equals "Walter Winchell"? false "Stephen Edwin King" equals "stephen edwin king"? true
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.equalsIgnoreCase() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 15: Tarea11_015 - Imprimir la Fecha y Hora Actual Formateada

Comprensión del Problema	
Descripción del problema:	Imprimir la Fecha y Hora Actual Formateada. Write a Java program to print

	the current date and time in the specified format.
Entradas requeridas:	Cadenas o parámetros de texto definidos en el método principal (main).
Salidas esperadas:	Current Date and Time : June 19, 2017 3:13 pm
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para imprimir la fecha y hora actual formateada utilizando operaciones de la clase String (Manipulación de cadenas y estructuras de control).
Variables identificadas(tipo y uso)	Variables de tipo String, int y char según los argumentos requeridos por la lógica.
Operaciones , Formulas y Reglas :	Métodos aplicados: Manipulación de cadenas y estructuras de control. Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	Current Date and Time : June 19, 2017 3:13 pm
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos Manipulación de cadenas y estructuras de control e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 16: Tarea11_016 - Obtener el Contenido de una Cadena como Arreglo de Bytes

Comprensión del Problema

Descripción del problema:	Obtener el Contenido de una Cadena como Arreglo de Bytes. Write a Java program to get the contents of a given string as a byte array.
Entradas requeridas:	String str = "This is a sample String." String new_str = new String(byte_arr)
Salidas esperadas:	The new String equals This is a sample String.
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para obtener el contenido de una cadena como arreglo de bytes utilizando operaciones de la clase String (String.getBytes()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"This is a sample String."' String new_str: Almacena la cadena de texto 'new String(byte_arr)'
Operaciones , Formulas y Reglas :	Métodos aplicados: String.getBytes(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The new String equals This is a sample String.
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.getBytes() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 17: Tarea11_017 - Obtener el Contenido de una Cadena como Arreglo de Caracteres

Comprensión del Problema	
Descripción del problema:	Obtener el Contenido de una Cadena como Arreglo de Caracteres. Write a

	Java program to get the contents of a given string as a character array.
Entradas requeridas:	String str = "This is a sample String."
Salidas esperadas:	The char array equals "[C@2a139a55"
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para obtener el contenido de una cadena como arreglo de caracteres utilizando operaciones de la clase String (String.toCharArray()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"This is a sample String."'
Operaciones , Formulas y Reglas :	Métodos aplicados: String.toCharArray(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The char array equals "[C@2a139a55"
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.toCharArray() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 18: Tarea11_018 - Generar un Código Hash Único para una Cadena

Comprensión del Problema	
Descripción del problema:	Generar un Código Hash Único para una Cadena. Write a Java program to create a distinct identifier for a given string.
Entradas requeridas:	String str = "Python Exercises."

	int hash_code = str.hashCode()
Salidas esperadas:	The hash for Python Exercises. is 863132599
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para generar un código hash único para una cadena utilizando operaciones de la clase String (String.hashCode()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"Python Exercises."' int hash_code: Valor numérico o índice (str.hashCode())
Operaciones , Formulas y Reglas :	Métodos aplicados: String.hashCode(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The hash for Python Exercises. is 863132599
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.hashCode() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 19: Tarea11_019 - Obtener el Primer Índice de Cada Carácter del Abecedario

Comprensión del Problema	
Descripción del problema:	Obtener el Primer Índice de Cada Carácter del Abecedario. Write a Java program to get the index of all the characters of the alphabet. Sample string of all alphabet: "The quick brown fox jumps over the lazy dog."
Entradas requeridas:	String str = "The quick brown fox jumps over the lazy dog."

Salidas esperadas:	a b c d e f g h i j ========== 36 10 7 40 2 16 42 1 6 20 k l m n o p q r s t ========== 8 35 22 14 12 23 4 11 24 31 u v w x y z ========== 5 27 13 18 38 37
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para obtener el primer índice de cada carácter del abecedario utilizando operaciones de la clase String (String.indexOf()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"The quick brown fox jumps over the lazy dog."'
Operaciones , Formulas y Reglas :	Métodos aplicados: String.indexOf(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	a b c d e f g h i j ========== 36 10 7 40 2 16 42 1 6 20 k l m n o p q r s t ========== 8 35 22 14 12 23 4 11 24 31 u v w x y z ========== 5 27 13 18 38 37
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.indexOf() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.

¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 20: Tarea11_020 - Obtener la Representación Canónica de una Cadena (intern)

Comprensión del Problema	
Descripción del problema:	Obtener la Representación Canónica de una Cadena (intern). Write a Java program to get the Canonical representation of the string object.
Entradas requeridas:	String str1 = "Java Exercises" String str2 = new StringBuffer("Java").append(" Exercises").toString()
Salidas esperadas:	str1 == str2? false str1 == str3? true
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para obtener la representación canónica de una cadena (intern) utilizando operaciones de la clase String (String.intern()).
Variables identificadas(tipo y uso)	String str1: Almacena la cadena de texto '"Java Exercises"' String str2: Almacena la cadena de texto 'new StringBuffer("Java").append(" Exercises").toString()'
Operaciones , Formulas y Reglas :	Métodos aplicados: String.intern(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	str1 == str2? false str1 == str3? true
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.intern() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.

¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 21: Tarea11_021 - Obtener el Último Índice de Cada Carácter del Abecedario

Comprensión del Problema	
Descripción del problema:	Obtener el Último Índice de Cada Carácter del Abecedario. Write a Java program to get the last index of a string within a string. Sample string of all alphabet: "The quick brown fox jumps over the lazy dog."
Entradas requeridas:	String str = "The quick brown fox jumps over the lazy dog."
Salidas esperadas:	a b c d e f g h i j ========== 36 10 7 40 33 16 42 32 6 20 k l m n o p q r s t ========== 8 35 22 14 41 23 4 29 24 31 u v w x y z ========== 21 27 13 18 38 37
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para obtener el último índice de cada carácter del abecedario utilizando operaciones de la clase String (String.lastIndexOf()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"The quick brown fox jumps over the lazy dog."'
Operaciones , Formulas y Reglas :	Métodos aplicados: String.lastIndexOf(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	a b c d e f g h i j ========== 36 10 7 40 33 16 42 32 6 20 k l m n o p q r s t ========== 8 35 22 14 41 23 4 29 24 31 u v w x y z ========== 21 27 13 18 38 37
Restricciones :	Ninguna adicional; validación de límites para evitar

	StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.lastIndexOf() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 22: Tarea11_022 - Calcular la Longitud Total de una Cadena de Texto

Comprensión del Problema	
Descripción del problema:	Calcular la Longitud Total de una Cadena de Texto. Write a Java program to get the length of a given string.
Entradas requeridas:	String str = "example.com" int len = str.length()
Salidas esperadas:	The string length of 'example.com' is: 11
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para calcular la longitud total de una cadena de texto utilizando operaciones de la clase String (String.length()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"example.com"' int len: Valor numérico o índice (str.length())
Operaciones , Formulas y Reglas :	Métodos aplicados: String.length(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The string length of 'example.com' is: 11
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.

Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.length() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 23: Tarea11_023 - Comparar Regiones Específicas entre Dos Cadenas

Comprensión del Problema	
Descripción del problema:	Comparar Regiones Específicas entre Dos Cadenas. Write a Java program to find out whether a region in the current string matches a region in another string.
Entradas requeridas:	String str1 = "Shanghai Houston Colorado Alexandria" String str2 = "Alexandria Colorado Houston Shanghai"
Salidas esperadas:	str1[0 - 7] == str2[28 - 35]? true str1[9 - 15] == str2[9 - 15]? false
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para comparar regiones específicas entre dos cadenas utilizando operaciones de la clase String (String.regionMatches()).
Variables identificadas(tipo y uso)	String str1: Almacena la cadena de texto '"Shanghai Houston Colorado Alexandria"' String str2: Almacena la cadena de texto '"Alexandria Colorado Houston Shanghai"'
Operaciones , Formulas y Reglas :	Métodos aplicados: String.regionMatches(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	str1[0 - 7] == str2[28 - 35]? true str1[9 - 15] == str2[9 - 15]? false

Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.regionMatches() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 24: Tarea11_024 - Reemplazar un Carácter Específico por Otro Carácter

Comprensión del Problema	
Descripción del problema:	Reemplazar un Carácter Específico por Otro Carácter. Write a Java program to replace a specified character with another character.
Entradas requeridas:	String str = "The quick brown fox jumps over the lazy dog."
Salidas esperadas:	Original string: The quick brown fox jumps over the lazy dog. New String: The quick brown fox jumps over the lazy fog.
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para reemplazar un carácter específico por otro carácter utilizando operaciones de la clase String (String.replace()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"The quick brown fox jumps over the lazy dog."'
Operaciones , Formulas y Reglas :	Métodos aplicados: String.replace(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	Original string: The quick brown fox jumps over the lazy dog. New String: The quick brown fox jumps over the lazy fog.
Restricciones :	Ninguna adicional; validación de límites para evitar

	StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.replace() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 25: Tarea11_025 - Reemplazar Subcadenas Coincidentes con una Expresión Regular

Comprensión del Problema	
Descripción del problema:	Reemplazar Subcadenas Coincidentes con una Expresión Regular. Write a Java program to replace each substring of a given string that matches the given regular expression with the given replacement. Sample string : "The quick brown fox jumps over the lazy dog." In the above string replace all the fox with cat.
Entradas requeridas:	String str = "The quick brown fox jumps over the lazy dog."
Salidas esperadas:	Original string: The quick brown fox jumps over the lazy dog. New String: The quick brown cat jumps over the lazy dog.
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para reemplazar subcadenas coincidentes con una expresión regular utilizando operaciones de la clase String (String.replaceAll()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"The quick brown fox jumps over the lazy dog."'
Operaciones , Formulas y Reglas :	Métodos aplicados: String.replaceAll(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	Original string: The quick brown fox jumps over the lazy dog.

	New String: The quick brown cat jumps over the lazy dog.
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.replaceAll() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 26: Tarea11_026 - Comprobar si una Cadena Comienza con un Prefijo Determinado

Comprensión del Problema	
Descripción del problema:	Comprobar si una Cadena Comienza con un Prefijo Determinado. Write a Java program to check whether a given string starts with another string.
Entradas requeridas:	String str1 = "Red is favorite color." String str2 = "Orange is also my favorite color." String startStr = "Red"
Salidas esperadas:	Red is favorite color. starts with Red? true Orange is also my favorite color. starts with Red? false
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para comprobar si una cadena comienza con un prefijo determinado utilizando operaciones de la clase String (String.startsWith()).
Variables identificadas(tipo y uso)	String str1: Almacena la cadena de texto '"Red is favorite color."' String str2: Almacena la cadena de texto '"Orange is also my favorite color."' String startStr: Almacena la cadena de texto '"Red"'
Operaciones , Formulas y Reglas :	Métodos aplicados: String.startsWith(). Indexación basada en cero.

	Inmutabilidad de objetos String.
Salidas esperadas:	Red is favorite color. starts with Red? true Orange is also my favorite color. starts with Red? false
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.startsWith() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 27: Tarea11_027 - Extraer una Subcadena a partir de Dos Posiciones Determinadas

Comprensión del Problema	
Descripción del problema:	Extraer una Subcadena a partir de Dos Posiciones Determinadas. Write a Java program to get a substring of a given string at two specified positions.
Entradas requeridas:	String str = "The quick brown fox jumps over the lazy dog."
Salidas esperadas:	old = The quick brown fox jumps over the lazy dog. new = brown fox jumps
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para extraer una subcadena a partir de dos posiciones determinadas utilizando operaciones de la clase String (String.substring()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"The quick brown fox jumps over the lazy dog."'
Operaciones , Formulas y Reglas :	Métodos aplicados: String.substring(). Indexación basada en cero.

	Inmutabilidad de objetos String.
Salidas esperadas:	old = The quick brown fox jumps over the lazy dog. new = brown fox jumps
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.substring() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 28: Tarea11_028 - Crear un Arreglo de Caracteres que Contenga la Cadena

Comprensión del Problema	
Descripción del problema:	Crear un Arreglo de Caracteres que Contenga la Cadena. Write a Java program to create a character array containing a string.
Entradas requeridas:	String str = "Java Exercises."
Salidas esperadas:	Java Exercises.
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para crear un arreglo de caracteres que contenga la cadena utilizando operaciones de la clase String (String.toCharArray()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"Java Exercises."'
Operaciones , Formulas y Reglas :	Métodos aplicados: String.toCharArray(). Indexación basada en cero. Inmutabilidad de objetos String.

Salidas esperadas:	Java Exercises.
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.toCharArray() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 29: Tarea11_029 - Convertir Todos los Caracteres de una Cadena a Minúsculas

Comprensión del Problema	
Descripción del problema:	Convertir Todos los Caracteres de una Cadena a Minúsculas. Write a Java program to convert all characters in a string to lowercase.
Entradas requeridas:	String str = "The Quick BroWn FoX!"
Salidas esperadas:	Original String: The Quick BroWn FoX! String in lowercase: the quick brown fox!
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para convertir todos los caracteres de una cadena a minúsculas utilizando operaciones de la clase String (String.toLowerCase()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"The Quick BroWn FoX!"'
Operaciones , Formulas y Reglas :	Métodos aplicados: String.toLowerCase(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	Original String: The Quick BroWn FoX! String in lowercase: the quick brown fox!

Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.toLowerCase() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 30: Tarea11_030 - Convertir Todos los Caracteres de una Cadena a Mayúsculas

Comprensión del Problema	
Descripción del problema:	Convertir Todos los Caracteres de una Cadena a Mayúsculas. Write a Java program to convert all characters in a string to uppercase.
Entradas requeridas:	String str = "The Quick BroWn FoX!"
Salidas esperadas:	Original String: The Quick BroWn FoX! String in uppercase: THE QUICK BROWN FOX!
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para convertir todos los caracteres de una cadena a mayúsculas utilizando operaciones de la clase String (String.toUpperCase()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"The Quick BroWn FoX!"'
Operaciones , Formulas y Reglas :	Métodos aplicados: String.toUpperCase(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	Original String: The Quick BroWn FoX! String in uppercase: THE QUICK BROWN FOX!
Restricciones :	Ninguna adicional; validación de límites para evitar

	StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.toUpperCase() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 31: Tarea11_031 - Eliminar Espacios en Blanco al Inicio y al Final (trim)

Comprensión del Problema	
Descripción del problema:	Eliminar Espacios en Blanco al Inicio y al Final (trim). Write a Java program to trim leading or trailing whitespace from a given string.
Entradas requeridas:	String str = " Java Exercises "
Salidas esperadas:	Original String: Java Exercises New String: Java Exercises
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para eliminar espacios en blanco al inicio y al final (trim) utilizando operaciones de la clase String (String.trim()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '" Java Exercises "'
Operaciones , Formulas y Reglas :	Métodos aplicados: String.trim(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	Original String: Java Exercises New String: Java Exercises
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.

Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.trim() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 32: Tarea11_032 - Encontrar la Subcadena Palindrómica Más Larga

Comprensión del Problema	
Descripción del problema:	Encontrar la Subcadena Palindrómica Más Larga. Write a Java program to find the longest Palindromic Substring within a string.
Entradas requeridas:	String str = "thequickbrownfoxxofnworbquickthe" int start = 0, end = 0 int len1 = expandAroundCenter(s, i, i)
Salidas esperadas:	The given string is: thequickbrownfoxxofnworbquickthe The longest palindrome substring in the giv en string is; brownfoxxofnworb The length of the palindromic substring is: 16
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para encontrar la subcadena palindrómica más larga utilizando operaciones de la clase String (String.charAt(), String.length(), String.substring()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"thequickbrownfoxxofnworbquickthe"' int start: Valor numérico o índice (0, end = 0) int len1: Valor numérico o índice (expandAroundCenter(s, i, i)) int len2: Valor numérico o índice (expandAroundCenter(s, i, i + 1))
Operaciones , Formulas y Reglas :	Métodos aplicados: String.charAt(), String.length(), String.substring().

	Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given string is: thequickbrownfoxxofnworbquickthe The longest palindrome substring in the giv en string is; brownfoxxofnworb The length of the palindromic substring is: 16
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.charAt(), String.length(), String.substring() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 33: Tarea11_033 - Generar Todos los Entrelazados (Interleavings) de Dos Cadenas

Comprensión del Problema	
Descripción del problema:	Generar Todos los Entrelazados (Interleavings) de Dos Cadenas. Write a Java program to find all interleavings of given strings.
Entradas requeridas:	String str1 = "WX" String str2 = "YZ"
Salidas esperadas:	The given strings are: WX YZ The interleavings strings are: YWZX WYZX YWXZ WXYZ YZWX WYXZ
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.

Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para generar todos los entrelazados (interleavings) de dos cadenas utilizando operaciones de la clase String (String.charAt(), String.length(), String.substring()).
Variables identificadas(tipo y uso)	String str1: Almacena la cadena de texto '"WX"' String str2: Almacena la cadena de texto '"YZ"'
Operaciones , Formulas y Reglas :	Métodos aplicados: String.charAt(), String.length(), String.substring(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given strings are: WX YZ The interleavings strings are: YWZX WYZX YWXZ WXYZ YZWX WYXZ
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.charAt(), String.length(), String.substring() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 34: Tarea11_034 - Encontrar el Segundo Carácter Más Frecuente en una Cadena

Comprensión del Problema	
Descripción del problema:	Encontrar el Segundo Carácter Más Frecuente en una Cadena. Write a Java program to find the second most frequent character in a given string.
Entradas requeridas:	String str = "successes"

	int first = 0, second = 0
Salidas esperadas:	The given string is: successes The second most frequent char in the string is: c
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para encontrar el segundo carácter más frecuente en una cadena utilizando operaciones de la clase String (String.charAt(), String.length()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"successes"' int first: Valor numérico o índice (0, second = 0)
Operaciones , Formulas y Reglas :	Métodos aplicados: String.charAt(), String.length(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given string is: successes The second most frequent char in the string is: c
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.charAt(), String.length() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 35: Tarea11_035 - Generar Todas las Permutaciones con Repetición de una Cadena

Comprensión del Problema	
Descripción del problema:	Generar Todas las Permutaciones con Repetición de una Cadena. Write a Java program to print all permutations of a given string with repetition.

Entradas requeridas:	String str = "PQR"
Salidas esperadas:	The given string is: PQR The permuted strings are: PPP PPQ PPR ... RRP RRQ RRR
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para generar todas las permutaciones con repetición de una cadena utilizando operaciones de la clase String (String.charAt(), String.length()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"PQR"'
Operaciones , Formulas y Reglas :	Métodos aplicados: String.charAt(), String.length(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given string is: PQR The permuted strings are: PPP PPQ PPR ... RRP RRQ RRR
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.charAt(), String.length() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos,

	coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 36: Tarea11_036 - Verificar si una Cadena es un Entrelazado Válido de Otras Dos

Comprensión del Problema	
Descripción del problema:	Verificar si una Cadena es un Entrelazado Válido de Otras Dos. Write a Java program to check whether two strings interlive of a given string. Assuming that unique characters are present in both strings.
Entradas requeridas:	String str1 = "PMQNO" String s1 = "MNO" String s2 = "PQ"
Salidas esperadas:	The given string is: PMQNO The interleaving strings are MNO and PQ The given string is interleaving: true The given string is: PNQMO The interleaving strings are MNO and PQ The given string is interleaving: false
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para verificar si una cadena es un entrelazado válido de otras dos utilizando operaciones de la clase String (String.charAt(), String.length()).
Variables identificadas(tipo y uso)	String str1: Almacena la cadena de texto '"PMQNO"' String s1: Almacena la cadena de texto '"MNO"' String s2: Almacena la cadena de texto '"PQ"' String str2: Almacena la cadena de texto '"PNQMO"'
Operaciones , Formulas y Reglas :	Métodos aplicados: String.charAt(), String.length(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given string is: PMQNO The interleaving strings are MNO and PQ The given string is interleaving: true The given string is: PNQMO The interleaving strings are MNO and PQ The given string is interleaving: false

Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.charAt(), String.length() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 37: Tarea11_037 - Longitud de la Subcadena Más Larga sin Caracteres Repetidos

Comprensión del Problema	
Descripción del problema:	Longitud de la Subcadena Más Larga sin Caracteres Repetidos. Write a Java program to find the length of the longest substring of a given string without repeating characters.
Entradas requeridas:	String str = "pickoutthelongestsubstring" String longest = "" char ch = str.charAt(j)
Salidas esperadas:	Input String : pickoutthelongestsubstring The longest substring : [u, b, s, t, r, i, n, g] The longest Substring Length : 8
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para longitud de la subcadena más larga sin caracteres repetidos utilizando operaciones de la clase String (String.charAt(), String.toCharArray(), String.length()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"pickoutthelongestsubstring"' String longest: Almacena la cadena de texto '""' char ch: Carácter objetivo (str.charAt(j))

Operaciones , Formulas y Reglas :	Métodos aplicados: String.charAt(), String.toCharArray(), String.length(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	Input String : pickoutthelongestsubstring The longest substring : [u, b, s, t, r, i, n, g] The longest Substring Length : 8
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.charAt(), String.toCharArray(), String.length() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 38: Tarea11_038 - Eliminar Todos los Caracteres Duplicados de una Cadena

Comprensión del Problema	
Descripción del problema:	Eliminar Todos los Caracteres Duplicados de una Cadena. Write a Java program to print the result of removing duplicates from a given string.
Entradas requeridas:	String str = "w3resource"
Salidas esperadas:	The given string is: w3resource After removing duplicates characters the new string is: w3resouc
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para eliminar todos los caracteres duplicados de una cadena utilizando operaciones de la clase String (String.toCharArray()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"w3resource"'

Operaciones , Formulas y Reglas :	Métodos aplicados: String.toCharArray(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given string is: w3resource After removing duplicates characters the new string is: w3resouc
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.toCharArray() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 39: Tarea11_039 - Encontrar el Primer Carácter No Repetido en una Cadena

Comprensión del Problema	
Descripción del problema:	Encontrar el Primer Carácter No Repetido en una Cadena. Write a Java program to find the first non-repeating character in a string.
Entradas requeridas:	String str = "gibblegabbler" char result = ' '
Salidas esperadas:	The given string is: gibblegabbler The first non repeated character in String is: i
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para encontrar el primer carácter no repetido en una cadena utilizando operaciones de la clase String (String.charAt(), String.length()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"gibblegabbler"' char result: Carácter objetivo (' ')

Operaciones , Formulas y Reglas :	Métodos aplicados: String.charAt(), String.length(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given string is: gibblegabbler The first non repeated character in String is: i
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.charAt(), String.length() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 40: Tarea11_040 - Dividir una Cadena de Texto en N Partes Iguales

Comprensión del Problema	
Descripción del problema:	Dividir una Cadena de Texto en N Partes Iguales. Write a Java program to divide a string into n equal parts.
Entradas requeridas:	String str = "abcdefghijklmnopqrstuvwxy" int n = 5 int str_size = str.length()
Salidas esperadas:	The given string is: abcdefghijklmnopqrstuvwxy The string divided into 5 parts and they are: abcde fghij klmno pqrst uvwxy
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	

¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para dividir una cadena de texto en n partes iguales utilizando operaciones de la clase String (String.charAt(), String.length()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"abcdefghijklmnopqrstuvwxy"' int n: Valor numérico o índice (5) int str_size: Valor numérico o índice (str.length())
Operaciones , Formulas y Reglas :	Métodos aplicados: String.charAt(), String.length(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given string is: abcdefghijklmnopqrstuvwxy The string divided into 5 parts and they are: abcde fghij klmno pqrst uvwxy
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.charAt(), String.length() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 41: Tarea11_041 - Eliminar Caracteres de una Cadena Presentes en una Máscara

Comprensión del Problema	
Descripción del problema:	Eliminar Caracteres de una Cadena Presentes en una Máscara. Write a Java program to remove duplicate characters from a given string that appear in another given string.
Entradas requeridas:	String str = "the quick brown fox" String mask = "queen"

Salidas esperadas:	The given string is: the quick brown fox The given mask string is: queen The new string is: th ick brow fox
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para eliminar caracteres de una cadena presentes en una máscara utilizando operaciones de la clase String (String.toCharArray()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"the quick brown fox"' String mask: Almacena la cadena de texto '"queen"'
Operaciones , Formulas y Reglas :	Métodos aplicados: String.toCharArray(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given string is: the quick brown fox The given mask string is: queen The new string is: th ick brow fox
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.toCharArray() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 42: Tarea11_042 - Listar Elementos que Contienen Todas las Letras de una Palabra

Comprensión del Problema

Descripción del problema:	Listar Elementos que Contienen Todas las Letras de una Palabra. Write a Java program to print a list of items containing all characters of a given word.
Entradas requeridas:	String word = "bib"
Salidas esperadas:	The given strings are: rabbit bribe dog The given word is: bib The strings containing all the letters of the given word are: rabbit bribe
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para listar elementos que contienen todas las letras de una palabra utilizando operaciones de la clase String (String.toCharArray()).
Variables identificadas(tipo y uso)	String word: Almacena la cadena de texto '"bib"'
Operaciones , Formulas y Reglas :	Métodos aplicados: String.toCharArray(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given strings are: rabbit bribe dog The given word is: bib The strings containing all the letters of the given word are: rabbit bribe
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.toCharArray() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 43: Tarea11_043 - Encontrar el Carácter con Mayor Frecuencia de Aparición

Comprensión del Problema	
Descripción del problema:	Encontrar el Carácter con Mayor Frecuencia de Aparición. Write a Java program to find the character in a string that occurs the most frequently.
Entradas requeridas:	String str = "test string" char maxChar = ' ' int max = -1
Salidas esperadas:	The given string is: test string Max occurring character in the given string is: t
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para encontrar el carácter con mayor frecuencia de aparición utilizando operaciones de la clase String (String.toCharArray()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"test string"' char maxChar: Carácter objetivo (' ') int max: Valor numérico o índice (-1)
Operaciones , Formulas y Reglas :	Métodos aplicados: String.toCharArray(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given string is: test string Max occurring character in the given string is: t
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.toCharArray() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 44: Tarea11_044 - Invertir una Cadena de Forma Recursiva

Comprensión del Problema	
Descripción del problema:	Invertir una Cadena de Forma Recursiva. Write a Java program to reverse a string using recursion.
Entradas requeridas:	String str = "The quick brown fox jumps"
Salidas esperadas:	The given string is: The quick brown fox jumps The string in reverse order is: spmuj xof nworb kciuq ehT
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para invertir una cadena de forma recursiva utilizando operaciones de la clase String (String.charAt(), String.length(), String.substring()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"The quick brown fox jumps"'
Operaciones , Formulas y Reglas :	Métodos aplicados: String.charAt(), String.length(), String.substring(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given string is: The quick brown fox jumps The string in reverse order is: spmuj xof nworb kciuq ehT
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.charAt(), String.length(), String.substring() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 45: Tarea11_045 - Invertir el Orden de las Palabras en una Cadena

Comprensión del Problema	
Descripción del problema:	Invertir el Orden de las Palabras en una Cadena. Write a Java program to reverse words in a given string.
Entradas requeridas:	String str = "Reverse words in a given string"
Salidas esperadas:	The given string is: Reverse words in a given string The new string after reversed the words: string given a in words Reverse
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para invertir el orden de las palabras en una cadena utilizando operaciones de la clase String (String.split()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"Reverse words in a given string"'
Operaciones , Formulas y Reglas :	Métodos aplicados: String.split(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given string is: Reverse words in a given string The new string after reversed the words: string given a in words Reverse
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.split() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 46: Tarea11_046 - Invertir Cada Palabra Individualmente Manteniendo su Posición

Comprensión del Problema	
Descripción del problema:	Invertir Cada Palabra Individualmente Manteniendo su Posición. Write a Java program to reverse every word in a string using methods.
Entradas requeridas:	String str = "This is a test string"
Salidas esperadas:	The given string is: This is a test string The string reversed word by word is: sihT si a tset gnirts
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para invertir cada palabra individualmente manteniendo su posición utilizando operaciones de la clase String (String.split(), String.reverse()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"This is a test string"'
Operaciones , Formulas y Reglas :	Métodos aplicados: String.split(), String.reverse(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given string is: This is a test string The string reversed word by word is: sihT si a tset gnirts
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.split(), String.reverse() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 47: Tarea11_047 - Reorganizar Cadena Distanciando Caracteres Idénticos

Comprensión del Problema	
Descripción del problema:	Reorganizar Cadena Distanciando Caracteres Idénticos. Write a Java program to rearrange a string so that the same characters are positioned a distance away.
Entradas requeridas:	String str = "accessories"
Salidas esperadas:	The given string is: accessories The string after arrange newly is: secrsecisao
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para reorganizar cadena distanciando caracteres idénticos utilizando operaciones de la clase String (String.compareTo(), String.toCharArray(), String.length()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"accessories"'
Operaciones , Formulas y Reglas :	Métodos aplicados: String.compareTo(), String.toCharArray(), String.length(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given string is: accessories The string after arrange newly is: secrsecisao
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.compareTo(), String.toCharArray(), String.length() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 48: Tarea11_048 - Eliminar las Ocurrencias de 'b' y 'ac' en una Cadena

Comprensión del Problema	
Descripción del problema:	Eliminar las Ocurrencias de 'b' y 'ac' en una Cadena. Write a Java program to remove "b" and "ac" from a given string.
Entradas requeridas:	String str = "abrambabasc"
Salidas esperadas:	The given string is: abrambabasc After removing the new string is: aramaasc
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para eliminar las ocurrencias de 'b' y 'ac' en una cadena utilizando operaciones de la clase String (String.charAt(), String.length()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"abrambabasc"'
Operaciones , Formulas y Reglas :	Métodos aplicados: String.charAt(), String.length(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given string is: abrambabasc After removing the new string is: aramaasc
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.charAt(), String.length() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 49: Tarea11_049 - Primer Carácter No Repetido en un Flujo Continuo (Stream)

Comprensión del Problema	
Descripción del problema:	Primer Carácter No Repetido en un Flujo Continuo (Stream). Write a Java program to find the first non-repeating character from a stream of characters.
Entradas requeridas:	String str = "godisgood" char ch = str.charAt(i)
Salidas esperadas:	String: godisgood Reading: g The first non-repeating character so far is: g Reading: o The first non-repeating character so far is: g Reading: d The first non-repeating character so far is: g Reading: i The first non-repeating character so far is: g Reading: s The first non-repeating character so far is: g Reading: g The first non-repeating character so far is: o Reading: o The first non-repeating character so far is: d Reading: o The first non-repeating character so far is: d Reading: d The first non-repeating character so far is: i
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para primer carácter no repetido en un flujo continuo (stream) utilizando operaciones de la clase String (String.charAt(), String.length()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"godisgood"' char ch: Carácter objetivo (str.charAt(i))
Operaciones , Formulas y Reglas :	Métodos aplicados: String.charAt(), String.length(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	String: godisgood Reading: g The first non-repeating character so far is: g Reading: o

	The first non-repeating character so far is: g Reading: d The first non-repeating character so far is: g Reading: i The first non-repeating character so far is: g Reading: s The first non-repeating character so far is: g Reading: g The first non-repeating character so far is: o Reading: o The first non-repeating character so far is: d Reading: o The first non-repeating character so far is: d Reading: d The first non-repeating character so far is: i
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.charAt(), String.length() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 50: Tarea11_050 - Calcular el Rango Lexicográfico de una Cadena

Comprensión del Problema	
Descripción del problema:	Calcular el Rango Lexicográfico de una Cadena. Write a Java program to find the lexicographic rank of a given string. N.B.: Total possible permutations of BDCA are(lexicographic order) : ABCD ABDC ACBD ACDB ADBC ADCB BACD BADC BCAD BCDA BDAC BDCA 1 2 3 4 5 6 7 8 9 10 11 12

	The BDCA appear in 12 position of permutation (lexicographic order).
Entradas requeridas:	String str = "BDCA" int len = str.length() int rank = 1
Salidas esperadas:	The Given String is: BDCA The Lexicographic rank of the given string is: 12
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para calcular el rango lexicográfico de una cadena utilizando operaciones de la clase String (String.charAt(), String.length()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"BDCA"' int len: Valor numérico o índice (str.length()) int rank: Valor numérico o índice (1) int count: Valor numérico o índice (0)
Operaciones , Formulas y Reglas :	Métodos aplicados: String.charAt(), String.length(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The Given String is: BDCA The Lexicographic rank of the given string is: 12
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.charAt(), String.length() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 51: Tarea11_051 - Contar e Imprimir Caracteres Duplicados y sus Ocurrencias

Comprensión del Problema	
Descripción del problema:	Contar e Imprimir Caracteres Duplicados y sus Ocurrencias. Write a Java program to count and print all duplicates in the input string.
Entradas requeridas:	String str = "w3resource"
Salidas esperadas:	The given string is: w3resource The duplicate characters and counts are: e appears 2 times r appears 2 times
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para contar e imprimir caracteres duplicados y sus ocurrencias utilizando operaciones de la clase String (String.toCharArray()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"w3resource"'
Operaciones , Formulas y Reglas :	Métodos aplicados: String.toCharArray(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given string is: w3resource The duplicate characters and counts are: e appears 2 times r appears 2 times
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.toCharArray() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 52: Tarea11_052 - Verificar si Dos Cadenas son Rotaciones una de la Otra

Comprensión del Problema	
Descripción del problema:	Verificar si Dos Cadenas son Rotaciones una de la Otra. Write a Java program to check if two given strings are rotations of each other.
Entradas requeridas:	String str1 = "ABACD" String str2 = "CDABA"
Salidas esperadas:	The given strings are: ABACD and CDABA The concatination of 1st string twice is: ABACDABACD The 2nd string CDABA exists in the new string. Strings are rotations of each other
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para verificar si dos cadenas son rotaciones una de la otra utilizando operaciones de la clase String (String.contains(), String.length()).
Variables identificadas(tipo y uso)	String str1: Almacena la cadena de texto '"ABACD"' String str2: Almacena la cadena de texto '"CDABA"'
Operaciones , Formulas y Reglas :	Métodos aplicados: String.contains(), String.length(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given strings are: ABACD and CDABA The concatination of 1st string twice is: ABACDABACD The 2nd string CDABA exists in the new string. Strings are rotations of each other
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.contains(), String.length() e impresión mediante System.out.println().

¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 53: Tarea11_053 - Coincidencia de Cadenas con Patrones y Comodines (*, ?)

Comprensión del Problema	
Descripción del problema:	Coincidencia de Cadenas con Patrones y Comodines (*, ?). Write a Java program to match two strings where one string contains wildcard characters.
Entradas requeridas:	String str = "abcdhgh" String pattern = "abc*d?*"
Salidas esperadas:	The given string is: abcdhgh The given pattern string is: abc*d?* The given pattern is matching.
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para coincidencia de cadenas con patrones y comodines (*, ?) utilizando operaciones de la clase String (String.charAt(), String.length()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"abcdhgh"' String pattern: Almacena la cadena de texto '"abc*d?*"'
Operaciones , Formulas y Reglas :	Métodos aplicados: String.charAt(), String.length(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given string is: abcdhgh The given pattern string is: abc*d?* The given pattern is matching.
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.

¿Qué operaciones serán necesarias?	Invocación de métodos String.charAt(), String.length() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 54: Tarea11_054 - Ventana Mínima que Contiene Todos los Caracteres de un Patrón

Comprensión del Problema	
Descripción del problema:	Ventana Mínima que Contiene Todos los Caracteres de un Patrón. Write a Java program to find the smallest window in a string containing all characters in another string.
Entradas requeridas:	String str = "welcome to w3resource" String pat = "tower" int count = map.size()
Salidas esperadas:	The given string is: welcome to w3resource Characters to find in the main sring are: tower The smallest window which contains the finding characters is : to w3re
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para ventana mínima que contiene todos los caracteres de un patrón utilizando operaciones de la clase String (String.charAt(), String.toCharArray(), String.length(), String.substring()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"welcome to w3resource"' String pat: Almacena la cadena de texto '"tower"' int count: Valor numérico o índice (map.size()) int left: Valor numérico o índice (0, minLen = Integer.MAX_VALUE, start = 0)
Operaciones , Formulas y Reglas :	Métodos aplicados: String.charAt(), String.toCharArray(), String.length(), String.substring(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given string is: welcome to w3resource Characters to find in the main sring are: tower The smallest window which contains the finding characters is : to w3re
Restricciones :	Ninguna adicional; validación de límites para evitar

	StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.charAt(), String.toCharArray(), String.length(), String.substring() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 55: Tarea11_055 - Eliminar Duplicados Adyacentes de Forma Recursiva

Comprensión del Problema	
Descripción del problema:	Eliminar Duplicados Adyacentes de Forma Recursiva. Write a Java program to remove all adjacent duplicates recursively from a given string.
Entradas requeridas:	String str = "aabaarbarccrabmq" char dup = str.charAt(i)
Salidas esperadas:	The given string is: aabaarbarccrabmq The new string after removing all adjacent duplicates is: brmq
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para eliminar duplicados adyacentes de forma recursiva utilizando operaciones de la clase String (String.charAt(), String.length()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"aabaarbarccrabmq"' char dup: Carácter objetivo (str.charAt(i))
Operaciones , Formulas y Reglas :	Métodos aplicados: String.charAt(), String.length(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given string is: aabaarbarccrabmq The new string after removing all adjacent duplicates is:

	brmq
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.charAt(), String.length() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 56: Tarea11_056 - Concatenar Dos Cadenas Omitiendo Duplicado en el Punto de Unión

Comprensión del Problema	
Descripción del problema:	Concatenar Dos Cadenas Omitiendo Duplicado en el Punto de Unión. Write a Java program that appends two strings, omitting one character if the concatenation creates double characters.
Entradas requeridas:	String str1 = "food" String str2 = "door"
Salidas esperadas:	The given strings are: food and door The string after concatination are: foodoor
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para concatenar dos cadenas omitiendo duplicado en el punto de unión utilizando operaciones de la clase String (String.charAt(), String.length(), String.substring()).
Variables identificadas(tipo y uso)	String str1: Almacena la cadena de texto '"food"' String str2: Almacena la cadena de texto '"door"'
Operaciones , Formulas y Reglas :	Métodos aplicados: String.charAt(), String.length(), String.substring(). Indexación basada en cero. Inmutabilidad de objetos String.

Salidas esperadas:	The given strings are: food and door The string after concatination are: foodoor
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.charAt(), String.length(), String.substring() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 57: Tarea11_057 - Intercambiar los Dos Últimos Caracteres de una Cadena

Comprensión del Problema	
Descripción del problema:	Intercambiar los Dos Últimos Caracteres de una Cadena. Write a Java program to create a string from a given string by swapping the last two characters of the given string. The string length must be two or more.
Entradas requeridas:	String str = "string"
Salidas esperadas:	The given strings is: string The string after swap last two characters are: strign
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para intercambiar los dos últimos caracteres de una cadena utilizando operaciones de la clase String (String.charAt(), String.length(), String.substring()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"string"'
Operaciones , Formulas y Reglas :	Métodos aplicados: String.charAt(), String.length(), String.substring(). Indexación basada en cero. Inmutabilidad de objetos String.

Salidas esperadas:	The given strings is: string The string after swap last two characters are: strign
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.charAt(), String.length(), String.substring() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 58: Tarea11_058 - Comprobar si una Cadena Termina con Dos Caracteres Específicos

Comprensión del Problema	
Descripción del problema:	Comprobar si una Cadena Termina con Dos Caracteres Específicos. Write a Java program to read a string and return true if it ends with a specified string of length 2.
Entradas requeridas:	String str1 = "string" String str2 = "strign" String target = "ng"
Salidas esperadas:	The given strings is: string The string containing ng at last: true The given strings is: strign The string containing ng at last: false
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para comprobar si una cadena termina con dos caracteres específicos utilizando operaciones de la clase String (String.endsWith()).

Variables identificadas(tipo y uso)	String str1: Almacena la cadena de texto '"string"' String str2: Almacena la cadena de texto '"strign"' String target: Almacena la cadena de texto '"ng"'
Operaciones , Formulas y Reglas :	Métodos aplicados: String.endsWith(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given strings is: string The string containing ng at last: true The given strings is: strign The string containing ng at last: false
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.endsWith() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 59: Tarea11_059 - Verificar si Inicia con 'red' o 'black' y Retornar el Color

Comprensión del Problema	
Descripción del problema:	Verificar si Inicia con 'red' o 'black' y Retornar el Color. Write a Java program to read a string. If the string begins with "red" or "black" return that color string, otherwise return the empty string.
Entradas requeridas:	String str = "blacksea" String color = ""
Salidas esperadas:	The given strings is: blacksea The string begins with the color: black
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.

Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para verificar si inicia con 'red' o 'black' y retornar el color utilizando operaciones de la clase String (String.startsWith()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"blacksea"' String color: Almacena la cadena de texto '""'
Operaciones , Formulas y Reglas :	Métodos aplicados: String.startsWith(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given strings is: blacksea The string begins with the color: black
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.startsWith() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 60: Tarea11_060 - Igualar Longitud de Cadenas Recortando la Mayor y Concatenar

Comprensión del Problema	
Descripción del problema:	Igualar Longitud de Cadenas Recortando la Mayor y Concatenar. Write a Java program to read two strings append them together and return the result. If the strings are different lengths, remove characters from the beginning of the longer string and make them equal lengths.
Entradas requeridas:	String str1 = "Welcome" String str2 = "home" int minLen = Math.min(str1.length(), str2.length())
Salidas esperadas:	The given strings is: Welcome and home

	The new string is: comehome
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para igualar longitud de cadenas recortando la mayor y concatenar utilizando operaciones de la clase String (String.length(), String.substring()).
Variables identificadas(tipo y uso)	String str1: Almacena la cadena de texto '"Welcome"' String str2: Almacena la cadena de texto '"home"' int minLen: Valor numérico o índice (Math.min(str1.length(), str2.length()))
Operaciones , Formulas y Reglas :	Métodos aplicados: String.length(), String.substring(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given strings is: Welcome and home The new string is: comehome
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.length(), String.substring() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 61: Tarea11_061 - Crear Subcadena Tomando los Primeros y Últimos N Caracteres

Comprensión del Problema	
Descripción del problema:	Crear Subcadena Tomando los Primeros y Últimos N Caracteres. Write a Java program to create a new string taking specified number of characters from first and last position of a given string.
Entradas requeridas:	String str = "Welcome"

	int n = 3
Salidas esperadas:	The given strings is: Welcome The given numbers is: 3 The new string is: Welome
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para crear subcadena tomando los primeros y últimos n caracteres utilizando operaciones de la clase String (String.length(), String.substring()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"Welcome"' int n: Valor numérico o índice (3)
Operaciones , Formulas y Reglas :	Métodos aplicados: String.length(), String.substring(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given strings is: Welcome The given numbers is: 3 The new string is: Welome
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.length(), String.substring() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 62: Tarea11_062 - Comprobar si 'good' Aparece en la Posición 0 o 1

Comprensión del Problema	
Descripción del problema:	Comprobar si 'good' Aparece en la Posición 0 o 1. Write a Java program to

	read a string and return true if "good" appears starting at index 0 or 1 in the given string.
Entradas requeridas:	String str = "goodboy"
Salidas esperadas:	The given strings is: goodboy The 'good' appear in the string is: true
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para comprobar si 'good' aparece en la posición 0 o 1 utilizando operaciones de la clase String (String.length(), String.startsWith(), String.substring()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"goodboy"'
Operaciones , Formulas y Reglas :	Métodos aplicados: String.length(), String.startsWith(), String.substring(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given strings is: goodboy The 'good' appear in the string is: true
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.length(), String.startsWith(), String.substring() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 63: Tarea11_063 - Verificar si los Primeros Dos Caracteres Coinciden con los Últimos

Comprensión del Problema	
Descripción del problema:	Verificar si los Primeros Dos Caracteres Coinciden con los Últimos. Write a

	Java program to check whether the first two characters appear at the end of a given string.
Entradas requeridas:	String str = "educated"
Salidas esperadas:	The given strings is: educated The first two characters appear in the last is: true
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para verificar si los primeros dos caracteres coinciden con los últimos utilizando operaciones de la clase String (String.equals(), String.length(), String.substring()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"educated"'
Operaciones , Formulas y Reglas :	Métodos aplicados: String.equals(), String.length(), String.substring(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given strings is: educated The first two characters appear in the last is: true
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.equals(), String.length(), String.substring() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 64: Tarea11_064 - Quitar Subcadena de Longitud 2 del Inicio si Coincide con el Final

Comprensión del Problema	
Descripción del problema:	Quitar Subcadena de Longitud 2 del Inicio si Coincide con el Final. Write a

	Java program to read a string. If a substring of length two appears at both its beginning and end, return a string without the substring at the beginning; otherwise, return the original string unchanged.
Entradas requeridas:	String str = "educated"
Salidas esperadas:	The given strings is: educated The new string is: ucated
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para quitar subcadena de longitud 2 del inicio si coincide con el final utilizando operaciones de la clase String (String.equals(), String.length(), String.substring()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"educated"'
Operaciones , Formulas y Reglas :	Métodos aplicados: String.equals(), String.length(), String.substring(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given strings is: educated The new string is: ucated
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.equals(), String.length(), String.substring() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 65: Tarea11_065 - Quitar Primer y Último Carácter si son Idénticos

Comprensión del Problema

Descripción del problema:	Quitar Primer y Último Carácter si son Idénticos. Write a Java program to read a given string and return the string without the first or last characters if they are the same, otherwise return the string without the characters.
Entradas requeridas:	String str = "testcricket"
Salidas esperadas:	The given strings is: testcricket The new string is: estcricke
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para quitar primer y último carácter si son idénticos utilizando operaciones de la clase String (String.charAt(), String.length(), String.substring()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"testcricket"'
Operaciones , Formulas y Reglas :	Métodos aplicados: String.charAt(), String.length(), String.substring(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given strings is: testcricket The new string is: estcricke
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.charAt(), String.length(), String.substring() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 66: Tarea11_066 - Quitar los Primeros Dos Caracteres Excepto si son 'g' o 'h'

Comprensión del Problema

Descripción del problema:	Quitar los Primeros Dos Caracteres Excepto si son 'g' o 'h'. Write a Java program to read a string and return it without the first two characters. Keep the first character if it is 'g' and keep the second character if it is 'h'.
Entradas requeridas:	Cadenas o parámetros de texto definidos en el método principal (main).
Salidas esperadas:	The given strings is: goat The new string is: gat he given strings is: photo The new string is: hoto The given strings is: ghost The new string is: ghost
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para quitar los primeros dos caracteres excepto si son 'g' o 'h' utilizando operaciones de la clase String (String.charAt(), String.length(), String.substring()).
Variables identificadas(tipo y uso)	Variables de tipo String, int y char según los argumentos requeridos por la lógica.
Operaciones , Formulas y Reglas :	Métodos aplicados: String.charAt(), String.length(), String.substring(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given strings is: goat The new string is: gat he given strings is: photo The new string is: hoto The given strings is: ghost The new string is: ghost
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.charAt(), String.length(), String.substring() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.

¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 67: Tarea11_067 - Quitar Primeros Dos Caracteres si Coinciden con un Carácter Objetivo

Comprensión del Problema	
Descripción del problema:	Quitar Primeros Dos Caracteres si Coinciden con un Carácter Objetivo. Write a Java program to read a string and remove the first two characters if one or both are equal to a specified character, otherwise leave them unchanged.
Entradas requeridas:	char target = 'o'
Salidas esperadas:	The given strings is: oocyte The new string is: cyte The given strings is: boat The new string is: bat The given strings is: own The new string is: wn
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para quitar primeros dos caracteres si coinciden con un carácter objetivo utilizando operaciones de la clase String (String.charAt(), String.length()).
Variables identificadas(tipo y uso)	char target: Carácter objetivo ('o')
Operaciones , Formulas y Reglas :	Métodos aplicados: String.charAt(), String.length(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given strings is: oocyte The new string is: cyte The given strings is: boat The new string is: bat The given strings is: own The new string is: wn
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente.

	3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.charAt(), String.length() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 68: Tarea11_068 - Eliminar un Carácter Específico y sus Vecinos Inmediatos

Comprensión del Problema	
Descripción del problema:	Eliminar un Carácter Específico y sus Vecinos Inmediatos. Write Java program to read a string and return after removing specified characters and their immediate left and right adjacent characters.
Entradas requeridas:	int len = str.length()
Salidas esperadas:	The given strings is: test#string The new string is: testring The given strings is: test##string The new string is: testring The given strings is: test#the#string The new string is: teshtring
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para eliminar un carácter específico y sus vecinos inmediatos utilizando operaciones de la clase String (String.charAt(), String.length()).
Variables identificadas(tipo y uso)	int len: Valor numérico o índice (str.length())
Operaciones , Formulas y Reglas :	Métodos aplicados: String.charAt(), String.length(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given strings is: test#string The new string is: testring The given strings is: test##string The new string is: testring The given strings is: test#the#string The new string is: teshtring

Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.charAt(), String.length() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 69: Tarea11_069 - Extraer la Subcadena Ubicada entre la Primera y Última Aparición de 'toast'

Comprensión del Problema	
Descripción del problema:	Extraer la Subcadena Ubicada entre la Primera y Última Aparición de 'toast'. Write a Java program to return the substring that is between the first and last appearance of the substring 'toast' in the given string, or return an empty string if the substring 'toast' does not exist.
Entradas requeridas:	String str = "sweettoastbuttertoast" String target = "toast" int first = str.indexOf(target)
Salidas esperadas:	The given strings is: sweettoastbuttertoast The new string is: butter
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para extraer la subcadena ubicada entre la primera y última aparición de 'toast' utilizando operaciones de la clase String (String.indexOf(), String.lastIndexOf(), String.length(), String.substring()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"sweettoastbuttertoast"' String target: Almacena la cadena de texto '"toast"' int first: Valor numérico o índice (str.indexOf(target))

	int last: Valor numérico o índice (str.lastIndexOf(target))
Operaciones , Formulas y Reglas :	Métodos aplicados: String.indexOf(), String.lastIndexOf(), String.length(), String.substring(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given strings is: sweettoastbuttertoast The new string is: butter
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.indexOf(), String.lastIndexOf(), String.length(), String.substring() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 70: Tarea11_070 - Comprobar el Balance PQ (cada 'p' seguida de una 'q')

Comprensión del Problema	
Descripción del problema:	Comprobar el Balance PQ (cada 'p' seguida de una 'q'). Write a Java program that checks if a string has pq-balance if it contains at least one 'q' directly after each ‘p’. But a 'q' before the 'p' invalidates pq-balance.
Entradas requeridas:	int lastP = str.lastIndexOf('p') int lastQ = str.lastIndexOf('q')
Salidas esperadas:	The given strings is: gfpmpnppqab The string is pq-balanced? true The given strings is: gfpmpnpqpab The string is pq-balanced? false
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	

¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para comprobar el balance pq (cada 'p' seguida de una 'q') utilizando operaciones de la clase String (String.lastIndexOf()).
Variables identificadas(tipo y uso)	int lastP: Valor numérico o índice (str.lastIndexOf('p')) int lastQ: Valor numérico o índice (str.lastIndexOf('q'))
Operaciones , Formulas y Reglas :	Métodos aplicados: String.lastIndexOf(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given strings is: gfpmpnppqab The string is pq-balanced? true The given strings is: gfpmpnpqpab The string is pq-balanced? false
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.lastIndexOf() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 71: Tarea11_071 - Verificar si Alguna de las Cadenas Aparece al Final de la Otra

Comprensión del Problema	
Descripción del problema:	Verificar si Alguna de las Cadenas Aparece al Final de la Otra. Write a Java program to check two given strings whether any of them appears at the end of the other string (ignore case sensitivity).
Entradas requeridas:	Cadenas o parámetros de texto definidos en el método principal (main).
Salidas esperadas:	The given strings are: xyz and pqrxyz Is one string appears at the end of other? true The given strings are: pqrxyz and xyz Is one string appears at the end of other? true

Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para verificar si alguna de las cadenas aparece al final de la otra utilizando operaciones de la clase String (String.endsWith(), String.toLowerCase()).
Variables identificadas(tipo y uso)	Variables de tipo String, int y char según los argumentos requeridos por la lógica.
Operaciones , Formulas y Reglas :	Métodos aplicados: String.endsWith(), String.toLowerCase(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given strings are: xyz and pqrxyz Is one string appears at the end of other? true The given strings are: pqrxyz and xyz Is one string appears at the end of other? true
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.endsWith(), String.toLowerCase() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 72: Tarea11_072 - Comprobar si la Cadena Contiene el Patrón 'p?p'

Comprensión del Problema	
Descripción del problema:	Comprobar si la Cadena Contiene el Patrón 'p?p'. Write a Java program to return true if a given string contains the string 'pop', but the middle 'o' also may contain another character.
Entradas requeridas:	Cadenas o parámetros de texto definidos en el método principal (main).

Salidas esperadas:	The given string is: dikchapop Is p?p appear in the given string? true The given string is: dikp$pdik Is p?p appear in the given string? true
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para comprobar si la cadena contiene el patrón 'p?p' utilizando operaciones de la clase String (String.charAt(), String.length()).
Variables identificadas(tipo y uso)	Variables de tipo String, int y char según los argumentos requeridos por la lógica.
Operaciones , Formulas y Reglas :	Métodos aplicados: String.charAt(), String.length(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given string is: dikchapop Is p?p appear in the given string? true The given string is: dikp$pdik Is p?p appear in the given string? true
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.charAt(), String.length() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 73: Tarea11_073 - Verificar si una Subcadena Aparece Justo Antes de un Punto (.)

Comprensión del Problema

Descripción del problema:	Verificar si una Subcadena Aparece Justo Antes de un Punto (.). Write a Java program to check whether a substring appears before a period(.) within a given string.
Entradas requeridas:	int dot = str.indexOf('.')
Salidas esperadas:	The given strings is: testabc.test Is 'abc' appear before period? true The given string is: test.abctest Is 'abc' appear before period? false
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para verificar si una subcadena aparece justo antes de un punto (.) utilizando operaciones de la clase String (String.endsWith(), String.indexOf(), String.substring()).
Variables identificadas(tipo y uso)	int dot: Valor numérico o índice (str.indexOf('.'))
Operaciones , Formulas y Reglas :	Métodos aplicados: String.endsWith(), String.indexOf(), String.substring(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given strings is: testabc.test Is 'abc' appear before period? true The given string is: test.abctest Is 'abc' appear before period? false
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.endsWith(), String.indexOf(), String.substring() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 74: Tarea11_074 - Comprobar si un Prefijo de Longitud N Aparece en otra Parte

Comprensión del Problema	
Descripción del problema:	Comprobar si un Prefijo de Longitud N Aparece en otra Parte. Write a Java program to check whether a prefix string created using the first specific character in a given string appears somewhere else in the string.
Entradas requeridas:	Cadenas o parámetros de texto definidos en el método principal (main).
Salidas esperadas:	The given strings is: MrsJemsmrsam The prefix string length is: 3 Is 'Mrs' appear else where in the string? false The given string is: MrsJemsMrsam The prefix string length is: 3 Is 'Mrs' appear else where in the string? true
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para comprobar si un prefijo de longitud n aparece en otra parte utilizando operaciones de la clase String (String.contains(), String.substring()).
Variables identificadas(tipo y uso)	Variables de tipo String, int y char según los argumentos requeridos por la lógica.
Operaciones , Formulas y Reglas :	Métodos aplicados: String.contains(), String.substring(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given strings is: MrsJemsmrsam The prefix string length is: 3 Is 'Mrs' appear else where in the string? false The given string is: MrsJemsMrsam The prefix string length is: 3 Is 'Mrs' appear else where in the string? true
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.contains(), String.substring() e impresión

	mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 75: Tarea11_075 - Comprobar si una Subcadena Aparece en el Centro Exacto

Comprensión del Problema	
Descripción del problema:	Comprobar si una Subcadena Aparece en el Centro Exacto. Write a Java program to check whether a given substring appears in the middle of another string. Here middle means the difference between the number of characters to the left and right of the given substring is not more than 1.
Entradas requeridas:	int idx = str.indexOf(sub) int left = idx int right = str.length() - (idx + sub.length())
Salidas esperadas:	The given string is: xxxabcxxxxx Is abc appear in middle? false The given string is: xxabcxxx Is abc appear in middle? true
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para comprobar si una subcadena aparece en el centro exacto utilizando operaciones de la clase String (String.indexOf(), String.length()).
Variables identificadas(tipo y uso)	int idx: Valor numérico o índice (str.indexOf(sub)) int left: Valor numérico o índice (idx) int right: Valor numérico o índice (str.length() - (idx + sub.length()))
Operaciones , Formulas y Reglas :	Métodos aplicados: String.indexOf(), String.length(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given string is: xxxabcxxxxx Is abc appear in middle? false The given string is: xxabcxxx Is abc appear in middle? true
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.

Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.indexOf(), String.length() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 76: Tarea11_076 - Contar Ocurrencias del Patrón 'life' o 'li?e'

Comprensión del Problema	
Descripción del problema:	Contar Ocurrencias del Patrón 'life' o 'li?e'. Write a Java program to count how many times the substring 'life' appears anywhere in a given string. Counting can also happen with the substring 'li?e', any character instead of 'f'.
Entradas requeridas:	String str = "liveonwildlife" int count = 0
Salidas esperadas:	The given string is: liveonwildlife The substring life or li?e appear number of times: 2
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para contar ocurrencias del patrón 'life' o 'li?e' utilizando operaciones de la clase String (Manipulación de cadenas y estructuras de control).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"liveonwildlife"' int count: Valor numérico o índice (0)
Operaciones , Formulas y Reglas :	Métodos aplicados: Manipulación de cadenas y estructuras de control. Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given string is: liveonwildlife The substring life or li?e appear number of times: 2

Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos Manipulación de cadenas y estructuras de control e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 77: Tarea11_077 - Repetir una Cadena N Veces Unida por un Separador

Comprensión del Problema	
Descripción del problema:	Repetir una Cadena N Veces Unida por un Separador. Write a Java program to add a string with a specific number of times separated by a substring.
Entradas requeridas:	String str = "try" String sep = "best" int times = 3
Salidas esperadas:	The given strings are: try and best Number to times to be repeat: 3 The new string is: trybesttrybesttry
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para repetir una cadena n veces unida por un separador utilizando operaciones de la clase String (Manipulación de cadenas y estructuras de control).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"try"' String sep: Almacena la cadena de texto '"best"' int times: Valor numérico o índice (3)
Operaciones , Formulas y Reglas :	Métodos aplicados: Manipulación de cadenas y estructuras de control.

	Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given strings are: try and best Number to times to be repeat: 3 The new string is: trybesttrybesttry
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos Manipulación de cadenas y estructuras de control e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 78: Tarea11_078 - Repetir los Últimos N Caracteres un Número Determinado de Veces

Comprensión del Problema	
Descripción del problema:	Repetir los Últimos N Caracteres un Número Determinado de Veces. Write a Java program to repeat a specific number of characters for a specific number of times from the last part of a given string.
Entradas requeridas:	String str = "string" int n = 3 int times = 3
Salidas esperadas:	The given string is: string The new string after repetition: inginging
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para repetir los últimos n caracteres un número determinado de veces utilizando operaciones de la clase String (String.length(), String.substring()).

Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"string"' int n: Valor numérico o índice (3) int times: Valor numérico o índice (3)
Operaciones , Formulas y Reglas :	Métodos aplicados: String.length(), String.substring(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given string is: string The new string after repetition: inginging
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.length(), String.substring() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 79: Tarea11_079 - Eliminar el Carácter Central en el Patrón 'z?g'

Comprensión del Problema	
Descripción del problema:	Eliminar el Carácter Central en el Patrón 'z?g'. Write a Java program to create a string from a given string. This is done after removing the 2nd character from the substring of length three starting with 'z' and ending with 'g' presents in the said string.
Entradas requeridas:	String str = "zzgkitandkatcaketoket"
Salidas esperadas:	The given string is: zzgkitandkatcaketoket The new string is: zgkitandkatcaketoket
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para eliminar el carácter central en el patrón

	'z?g' utilizando operaciones de la clase String (String.charAt(), String.length()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"zzgkitandkatcaketoket"'
Operaciones , Formulas y Reglas :	Métodos aplicados: String.charAt(), String.length(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given string is: zzgkitandkatcaketoket The new string is: zgkitandkatcaketoket
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.charAt(), String.length() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 80: Tarea11_080 - Comprobar si un Carácter está Rodeado por Caracteres Idénticos

Comprensión del Problema	
Descripción del problema:	Comprobar si un Carácter está Rodeado por Caracteres Idénticos. Write a Java program to check whether the character immediately before and after a specified character is the same in a given string.
Entradas requeridas:	Cadenas o parámetros de texto definidos en el método principal (main).
Salidas esperadas:	The given string is: moon#night The before and after character are same: true The given string is: bat##ball The before and after character are same: false The given string is: #moon#night The before and after character are same: true
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.

Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para comprobar si un carácter está rodeado por caracteres idénticos utilizando operaciones de la clase String (String.charAt(), String.length()).
Variables identificadas(tipo y uso)	Variables de tipo String, int y char según los argumentos requeridos por la lógica.
Operaciones , Formulas y Reglas :	Métodos aplicados: String.charAt(), String.length(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given string is: moon#night The before and after character are same: true The given string is: bat##ball The before and after character are same: false The given string is: #moon#night The before and after character are same: true
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.charAt(), String.length() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 81: Tarea11_081 - Comprobar si 'red' y 'blue' Aparecen la Misma Cantidad de Veces

Comprensión del Problema	
Descripción del problema:	Comprobar si 'red' y 'blue' Aparecen la Misma Cantidad de Veces. Write a Java program to check whether two strings of length 3 and 4 appear the same number of times in a given string.
Entradas requeridas:	String str = "redcapmanwithbluecar" int redCount = countOccurrences(str, "red")

	int blueCount = countOccurrences(str, "blue")
Salidas esperadas:	The given string is: redcapmanwithbluecar The appearance of red and blue are same: true
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para comprobar si 'red' y 'blue' aparecen la misma cantidad de veces utilizando operaciones de la clase String (String.indexOf(), String.length()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"redcapmanwithbluecar"' int redCount: Valor numérico o índice (countOccurrences(str, "red")) int blueCount: Valor numérico o índice (countOccurrences(str, "blue")) int count: Valor numérico o índice (0, idx = 0)
Operaciones , Formulas y Reglas :	Métodos aplicados: String.indexOf(), String.length(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given string is: redcapmanwithbluecar The appearance of red and blue are same: true
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.indexOf(), String.length() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 82: Tarea11_082 - Duplicar Cada Carácter de una Cadena

Comprensión del Problema	
Descripción del problema:	Duplicar Cada Carácter de una Cadena. Write a Java program to create a

	string containing every character twice of a given string.
Entradas requeridas:	String str = "welcome"
Salidas esperadas:	The given string is: welcome The new string is: wweellccoommee
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para duplicar cada carácter de una cadena utilizando operaciones de la clase String (String.toCharArray()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"welcome"'
Operaciones , Formulas y Reglas :	Métodos aplicados: String.toCharArray(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given string is: welcome The new string is: wweellccoommee
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.toCharArray() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 83: Tarea11_083 - Intercalar Alternadamente los Caracteres de Dos Cadenas

Comprensión del Problema	
Descripción del problema:	Intercalar Alternadamente los Caracteres de Dos Cadenas. Write a Java program to create a string from two given strings. This is so that each character of the two strings appears individually in the created string.

Entradas requeridas:	String str1 = "welcome" String str2 = "w3resource" int maxLen = Math.max(str1.length(), str2.length())
Salidas esperadas:	The given strings are: welcome and w3resource The new string is: wwe3lrceosmoeurce
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para intercalar alternadamente los caracteres de dos cadenas utilizando operaciones de la clase String (String.charAt(), String.length()).
Variables identificadas(tipo y uso)	String str1: Almacena la cadena de texto '"welcome"' String str2: Almacena la cadena de texto '"w3resource"' int maxLen: Valor numérico o índice (Math.max(str1.length(), str2.length()))
Operaciones , Formulas y Reglas :	Métodos aplicados: String.charAt(), String.length(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given strings are: welcome and w3resource The new string is: wwe3lrceosmoeurce
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.charAt(), String.length() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 84: Tarea11_084 - Concatenar Prefijos Truncados en Orden Descendente

Comprensión del Problema

Descripción del problema:	Concatenar Prefijos Truncados en Orden Descendente. Write a Java program to make an original string made of p number of characters from the first character in a given string. This is followed by p-1 number of characters till p is greater than zero.
Entradas requeridas:	String str = "welcome" int p = 4
Salidas esperadas:	The given string is: welcome Number of repetition characters and repetition: 4 The new string is: welcwelwew
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para concatenar prefijos truncados en orden descendente utilizando operaciones de la clase String (String.substring()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"welcome"' int p: Valor numérico o índice (4)
Operaciones , Formulas y Reglas :	Métodos aplicados: String.substring(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given string is: welcome Number of repetition characters and repetition: 4 The new string is: welcwelwew
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.substring() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 85: Tarea11_085 - Extraer los Caracteres Inmediatamente Anteriores y Posteriores a una Subcadena

Comprensión del Problema	
Descripción del problema:	Extraer los Caracteres Inmediatamente Anteriores y Posteriores a una Subcadena. Write a Java program to make up a string with each character before and after a non-empty substring whichever it appears in a non-empty given string.
Entradas requeridas:	String str = "weablcoabmeab" String target = "ab" int idx = 0
Salidas esperadas:	The given string are: weablcoabmeab and ab The new string is: elome
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para extraer los caracteres inmediatamente anteriores y posteriores a una subcadena utilizando operaciones de la clase String (String.charAt(), String.indexOf(), String.length()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"weablcoabmeab"' String target: Almacena la cadena de texto '"ab"' int idx: Valor numérico o índice (0)
Operaciones , Formulas y Reglas :	Métodos aplicados: String.charAt(), String.indexOf(), String.length(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given string are: weablcoabmeab and ab The new string is: elome
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.charAt(), String.indexOf(), String.length() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos,

	coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 86: Tarea11_086 - Contar Tripletes de Caracteres Idénticos Consecutivos

Comprensión del Problema	
Descripción del problema:	Contar Tripletes de Caracteres Idénticos Consecutivos. Write a Java program to count the number of triples (characters appearing three times in a row) in a given string.
Entradas requeridas:	String str = "welllcommmmeee" int count = 0
Salidas esperadas:	The given string is: welllcommmmeee The number of triples in the string is: 4
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para contar tripletes de caracteres idénticos consecutivos utilizando operaciones de la clase String (String.charAt(), String.length()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"welllcommmmeee"' int count: Valor numérico o índice (0)
Operaciones , Formulas y Reglas :	Métodos aplicados: String.charAt(), String.length(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given string is: welllcommmmeee The number of triples in the string is: 4
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.charAt(), String.length() e impresión mediante System.out.println().

¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 87: Tarea11_087 - Comprobar si un Carácter es Feliz (rodeado por su mismo carácter)

Comprensión del Problema	
Descripción del problema:	Comprobar si un Carácter es Feliz (rodeado por su mismo carácter). Write a Java program to check whether a specified character is happy or not. A character is happy when the same character appears to its left or right in a string.
Entradas requeridas:	Cadenas o parámetros de texto definidos en el método principal (main).
Salidas esperadas:	The given string is: azzlea Is z happy in the string: true The given string is: azmzlea Is z happy in the string: falses
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para comprobar si un carácter es feliz (rodeado por su mismo carácter) utilizando operaciones de la clase String (String.charAt(), String.length()).
Variables identificadas(tipo y uso)	Variables de tipo String, int y char según los argumentos requeridos por la lógica.
Operaciones , Formulas y Reglas :	Métodos aplicados: String.charAt(), String.length(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given string is: azzlea Is z happy in the string: true The given string is: azmzlea Is z happy in the string: falses
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos.

	2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.charAt(), String.length() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 88: Tarea11_088 - Reemplazar la Palabra 'is' por 'is not'

Comprensión del Problema	
Descripción del problema:	Reemplazar la Palabra 'is' por 'is not'. Write a Java program to return a string where every appearance of the lowercase word 'is' has been replaced with 'is not'.
Entradas requeridas:	String str = "it is a string"
Salidas esperadas:	The given string is: it is a string The new string is: it is not a string
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para reemplazar la palabra 'is' por 'is not' utilizando operaciones de la clase String (String.replaceAll()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"it is a string"'
Operaciones , Formulas y Reglas :	Métodos aplicados: String.replaceAll(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given string is: it is a string The new string is: it is not a string
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente.

	3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.replaceAll() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 89: Tarea11_089 - Sumar Todos los Números Enteros Presentes en una Cadena

Comprensión del Problema	
Descripción del problema:	Sumar Todos los Números Enteros Presentes en una Cadena. Write a Java program to calculate the sum of the numbers that appear in a given string.
Entradas requeridas:	String str = "it 15 is25 a 20string" int sum = 0
Salidas esperadas:	The given string is: it 15 is25 a 20string The sum of numbers in the string is: 60
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para sumar todos los números enteros presentes en una cadena utilizando operaciones de la clase String (Manipulación de cadenas y estructuras de control).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"it 15 is25 a 20string"' int sum: Valor numérico o índice (0)
Operaciones , Formulas y Reglas :	Métodos aplicados: Manipulación de cadenas y estructuras de control. Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given string is: it 15 is25 a 20string The sum of numbers in the string is: 60
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos.

	2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos Manipulación de cadenas y estructuras de control e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 90: Tarea11_090 - Comparar Frecuencia de Ocurrencias de Dos Subcadenas ('the' e 'is')

Comprensión del Problema	
Descripción del problema:	Comparar Frecuencia de Ocurrencias de Dos Subcadenas ('the' e 'is'). Write a Java program to check the number of times the two substrings appearing anywhere in a string.
Entradas requeridas:	String str = "Thisisthethesis" int countThe = count(str, "the") int countIs = count(str, "is")
Salidas esperadas:	The given string is: Thisisthethesis Are the appearance of 'the' and 'is' equal? false
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para comparar frecuencia de ocurrencias de dos subcadenas ('the' e 'is') utilizando operaciones de la clase String (String.indexOf()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"Thisisthethesis"' int countThe: Valor numérico o índice (count(str, "the")) int countIs: Valor numérico o índice (count(str, "is")) int cnt: Valor numérico o índice (0, idx = 0)
Operaciones , Formulas y Reglas :	Métodos aplicados: String.indexOf(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given string is: Thisisthethesis Are the appearance of 'the' and 'is' equal? false
Restricciones :	Ninguna adicional; validación de límites para evitar

	StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.indexOf() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 91: Tarea11_091 - Contar Palabras que Terminan en 'm' o 'n'

Comprensión del Problema	
Descripción del problema:	Contar Palabras que Terminan en 'm' o 'n'. Write a Java program to count the number of words ending in 'm' or 'n' (not case sensitive) in a given text.
Entradas requeridas:	String str = "mam is in the room" int count = 0
Salidas esperadas:	The given string is: mam is in the room The number of words ends eith m or n is: 3
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para contar palabras que terminan en 'm' o 'n' utilizando operaciones de la clase String (String.endsWith(), String.toLowerCase(), String.split()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"mam is in the room"' int count: Valor numérico o índice (0)
Operaciones , Formulas y Reglas :	Métodos aplicados: String.endsWith(), String.toLowerCase(), String.split(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given string is: mam is in the room The number of words ends eith m or n is: 3

Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.endsWith(), String.toLowerCase(), String.split() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 92: Tarea11_092 - Eliminar Todas las Instancias de una Subcadena Específica

Comprensión del Problema	
Descripción del problema:	Eliminar Todas las Instancias de una Subcadena Específica. Write a Java program that returns a substring after removing all instances of remove string as given from the given main string.
Entradas requeridas:	String mainStr = "This is the test string" String removeStr = "st"
Salidas esperadas:	The main string is: This is the test string The removable string is: st The new string is: This is the te ring
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para eliminar todas las instancias de una subcadena específica utilizando operaciones de la clase String (String.replace()).
Variables identificadas(tipo y uso)	String mainStr: Almacena la cadena de texto '"This is the test string"' String removeStr: Almacena la cadena de texto '"st"'
Operaciones , Formulas y Reglas :	Métodos aplicados: String.replace(). Indexación basada en cero. Inmutabilidad de objetos String.

Salidas esperadas:	The main string is: This is the test string The removable string is: st The new string is: This is the te ring
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.replace() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 93: Tarea11_093 - Subcadena Más Larga que es Prefijo y Sufijo Simultáneamente

Comprensión del Problema	
Descripción del problema:	Subcadena Más Larga que es Prefijo y Sufijo Simultáneamente. Write a Java program to find the longest substring that appears at both ends of a given string.
Entradas requeridas:	String str = "playersplay" String longest = ""
Salidas esperadas:	The given string is: playersplay The longest substring in the string is: play
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para subcadena más larga que es prefijo y sufijo simultáneamente utilizando operaciones de la clase String (String.equals(), String.length(), String.substring()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"playersplay"' String longest: Almacena la cadena de texto '""'

Operaciones , Formulas y Reglas :	Métodos aplicados: String.equals(), String.length(), String.substring(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given string is: playersplay The longest substring in the string is: play
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.equals(), String.length(), String.substring() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 94: Tarea11_094 - Subcadena Espejo Más Larga en Ambos Extremos de una Cadena

Comprensión del Problema	
Descripción del problema:	Subcadena Espejo Más Larga en Ambos Extremos de una Cadena. Write a Java program to find the longest mirror image string at both ends of a given string.
Entradas requeridas:	String str = "rotavator" int len = str.length()
Salidas esperadas:	The given string is: rotavator The longest mirror image string in the string is: rotavator
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para subcadena espejo más larga en ambos extremos de una cadena utilizando operaciones de la clase String (String.charAt(), String.length()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"rotavator"'

	int len: Valor numérico o índice (str.length())
Operaciones , Formulas y Reglas :	Métodos aplicados: String.charAt(), String.length(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given string is: rotavator The longest mirror image string in the string is: rotavator
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.charAt(), String.length() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 95: Tarea11_095 - Sumar los Dígitos Numéricos Individuales Presentes en la Cadena

Comprensión del Problema	
Descripción del problema:	Sumar los Dígitos Numéricos Individuales Presentes en la Cadena. Write a Java program to return the sum of the digits present in the given string. In the absence of digits, the sum is 0.
Entradas requeridas:	String str = "ab5c2d4ef12s" int sum = 0 char c = str.charAt(i)
Salidas esperadas:	The given string is: ab5c2d4ef12s The sum of the digits in the string is: 14
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para sumar los dígitos numéricos individuales presentes en la cadena utilizando operaciones de la clase String

	(String.charAt(), String.length()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"ab5c2d4ef12s"' int sum: Valor numérico o índice (0) char c: Carácter objetivo (str.charAt(i))
Operaciones , Formulas y Reglas :	Métodos aplicados: String.charAt(), String.length(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given string is: ab5c2d4ef12s The sum of the digits in the string is: 14
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.charAt(), String.length() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 96: Tarea11_096 - Eliminar un Carácter Excepto en la Primera y Última Posición

Comprensión del Problema	
Descripción del problema:	Eliminar un Carácter Excepto en la Primera y Última Posición. Write a Java program to create a new string after removing a specified character from a given string. This is except the first and last position.
Entradas requeridas:	String str = "zebrazone" char target = 'z'
Salidas esperadas:	The given string is: zebrazone The new string is: zebraone
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	

¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para eliminar un carácter excepto en la primera y última posición utilizando operaciones de la clase String (String.charAt(), String.length()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"zebrazone"' char target: Carácter objetivo ('z')
Operaciones , Formulas y Reglas :	Métodos aplicados: String.charAt(), String.length(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given string is: zebrazone The new string is: zebraone
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.charAt(), String.length() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 97: Tarea11_097 - Extraer Caracteres en Grupos de 3 Saltando 2 Posiciones

Comprensión del Problema	
Descripción del problema:	Extraer Caracteres en Grupos de 3 Saltando 2 Posiciones. Write a Java program to return a string with characters at index positions 0,1,2,5,6,7, ... from a given string. 98. Write a Java program to return the number of index positions from two given strings where they contain the same substring of two characters.
Entradas requeridas:	String str = "w3resource.com" int end = Math.min(i + 3, str.length())
Salidas esperadas:	The given string is: w3resource.com The new string is: w3rour.co
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1].

	Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para extraer caracteres en grupos de 3 saltando 2 posiciones utilizando operaciones de la clase String (String.length(), String.substring()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"w3resource.com"' int end: Valor numérico o índice (Math.min(i + 3, str.length()))
Operaciones , Formulas y Reglas :	Métodos aplicados: String.length(), String.substring(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given string is: w3resource.com The new string is: w3rour.co
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.length(), String.substring() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 98: Tarea11_098 - Comprobar si la Primera Ocurrencia de un Carácter está Duplicada

Comprensión del Problema	
Descripción del problema:	Comprobar si la Primera Ocurrencia de un Carácter está Duplicada. Write a Java program to check whether the first instance of a given character is immediately followed by the same character in a given string.
Entradas requeridas:	String str = "fizzez" char ch = 'z' int idx = str.indexOf(ch)
Salidas esperadas:	The given string is: fizzez

	Is 'z' appear twice respectively? true
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para comprobar si la primera ocurrencia de un carácter está duplicada utilizando operaciones de la clase String (String.charAt(), String.indexOf(), String.length()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"fizzez"' char ch: Carácter objetivo ('z') int idx: Valor numérico o índice (str.indexOf(ch))
Operaciones , Formulas y Reglas :	Métodos aplicados: String.charAt(), String.indexOf(), String.length(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given string is: fizzez Is 'z' appear twice respectively? true
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.charAt(), String.indexOf(), String.length() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 99: Tarea11_099 - Extraer Únicamente los Caracteres en Posiciones Pares

Comprensión del Problema	
Descripción del problema:	Extraer Únicamente los Caracteres en Posiciones Pares. Write a Java program to return an updated string using every character of even position from a given string.
Entradas requeridas:	String str = "w3resource.com"

Salidas esperadas:	The given string is: w3resource.com The new string is: wrsuc.o
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para extraer únicamente los caracteres en posiciones pares utilizando operaciones de la clase String (String.charAt(), String.length()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"w3resource.com"'
Operaciones , Formulas y Reglas :	Métodos aplicados: String.charAt(), String.length(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	The given string is: w3resource.com The new string is: wrsuc.o
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.charAt(), String.length() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 100: Tarea11_100 - Verificar Presencia de Subcadenas en un Texto Extenso

Comprensión del Problema	
Descripción del problema:	Verificar Presencia de Subcadenas en un Texto Extenso. Write a Java program to check if a given string contains another string. Returns true or false.
Entradas requeridas:	String original = "Java is the foundation for virtually every type of networked application and is the global standard for developing and delivering embedded and mobile applications, games, Web-based content, and

	enterprise software. With more than 9 million developers worldwide, Java enables you to efficiently develop, deploy and use exciting applications and services." String word1 = "million" String word2 = "millions"
Salidas esperadas:	Original string: Java is the foundation for virtually every type of networked application and is the global standard for developing and delivering embedded and mobile applications, games, Web-based content, and enterprise software. With more than 9 million developers worldwide, Java enables you to efficiently develop, deploy and use exciting applications and services. Is 'million' present in the said text? true Is 'millions' present in the said text? false
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para verificar presencia de subcadenas en un texto extenso utilizando operaciones de la clase String (String.contains()).
Variables identificadas(tipo y uso)	String original: Almacena la cadena de texto '"Java is the foundation for virtually every type of networked application and is the global standard for developing and delivering embedded and mobile applications, games, Web-based content, and enterprise software. With more than 9 million developers worldwide, Java enables you to efficiently develop, deploy and use exciting applications and services."' String word1: Almacena la cadena de texto '"million"' String word2: Almacena la cadena de texto '"millions"'
Operaciones , Formulas y Reglas :	Métodos aplicados: String.contains(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	Original string: Java is the foundation for virtually every type of networked application and is the global standard for developing and delivering embedded and mobile applications, games, Web-based content, and enterprise software. With more than 9 million developers worldwide, Java enables you to efficiently develop, deploy and use exciting applications and services. Is 'million' present in the said text? true Is 'millions' present in the said text? false
Restricciones :	Ninguna adicional; validación de límites para evitar

	StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.contains() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 101: Tarea11_101 - Comprobar si una Cadena Contiene Únicamente Dígitos Numéricos

Comprensión del Problema	
Descripción del problema:	Comprobar si una Cadena Contiene Únicamente Dígitos Numéricos. Write a Java program to test if a string contains only digits. Returns true or false.
Entradas requeridas:	String str1 = "131231231231231231231231231212312312" String str2 = "1312312312Z123123123123123123212312312"
Salidas esperadas:	First string: 131231231231231231231231231212312312 true Second string: 1312312312Z123123123123123123212312312 false
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para comprobar si una cadena contiene únicamente dígitos numéricos utilizando operaciones de la clase String (String.matches()).
Variables identificadas(tipo y uso)	String str1: Almacena la cadena de texto '"131231231231231231231231231212312312"' String str2: Almacena la cadena de texto '"1312312312Z123123123123123123212312312"'

Operaciones , Formulas y Reglas :	Métodos aplicados: String.matches(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	First string: 131231231231231231231231231212312312 true Second string: 13123123123Z123123123123123121231231 2 false
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.matches() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 102: Tarea11_102 - Convertir Cadena a Tipos Numéricos (int, long, float, double)

Comprensión del Problema	
Descripción del problema:	Convertir Cadena a Tipos Numéricos (int, long, float, double). Write a Java program to convert a given string to int, long, floating and double.
Entradas requeridas:	String strInt = "1323" int iVal = Integer.parseInt(strInt) String strLong = "13625478965325"
Salidas esperadas:	Convert String to int/Integer: "1323" as int is 1323 and as Integer is 1323 Convert String to long/Long: "13625478965325" as long is 13625478965325 and as Long is 13625478965325 Convert String to float/Float: "25.135F" as float is 25.135 and as Float is 25.135 Convert String to double/Double:

	"21.25478254D" as double is 21.25478254 and as Double is 21.25478254 false
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para convertir cadena a tipos numéricos (int, long, float, double) utilizando operaciones de la clase String (Manipulación de cadenas y estructuras de control).
Variables identificadas(tipo y uso)	String strInt: Almacena la cadena de texto '"1323"' int iVal: Valor numérico o índice (Integer.parseInt(strInt)) String strLong: Almacena la cadena de texto '"13625478965325"' String strFloat: Almacena la cadena de texto '"25.135F"'
Operaciones , Formulas y Reglas :	Métodos aplicados: Manipulación de cadenas y estructuras de control. Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	Convert String to int/Integer: "1323" as int is 1323 and as Integer is 1323 Convert String to long/Long: "13625478965325" as long is 13625478965325 and as Long is 13625478965325 Convert String to float/Float: "25.135F" as float is 25.135 and as Float is 25.135 Convert String to double/Double: "21.25478254D" as double is 21.25478254 and as Double is 21.25478254 false
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos Manipulación de cadenas y estructuras de control e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 103: Tarea11_103 - Eliminar Todas las Ocurrencias de un Carácter Específico

Comprensión del Problema	
Descripción del problema:	Eliminar Todas las Ocurrencias de un Carácter Específico. Write a Java program to remove a specified character from a given string.
Entradas requeridas:	String str = "abcdefabcdeabcdaaa" char ch = 'a'
Salidas esperadas:	Original string: abcdefabcdeabcdaaa Second string: bcdefbcdebcd
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para eliminar todas las ocurrencias de un carácter específico utilizando operaciones de la clase String (String.replace()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"abcdefabcdeabcdaaa"' char ch: Carácter objetivo ('a')
Operaciones , Formulas y Reglas :	Métodos aplicados: String.replace(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	Original string: abcdefabcdeabcdaaa Second string: bcdefbcdebcd
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.replace() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se	Variables temporales numéricas (índices/contadores), subcadenas

generarán?	intermedias o booleanos de resultado.

Ejercicio 104: Tarea11_104 - Ordenar un Arreglo de Cadenas por su Longitud

Comprensión del Problema	
Descripción del problema:	Ordenar un Arreglo de Cadenas por su Longitud. Write a Java program to sort in ascending and descending order by the length of the given array of strings.
Entradas requeridas:	Cadenas o parámetros de texto definidos en el método principal (main).
Salidas esperadas:	Original unsorted colors: [Green, White, Black, Pink, Orange, Blue, Champagne, Indigo, Ivory] Sorted color (descending order): [Champagne, Orange, Indigo, Green, White, Black, Ivory, Pink, Blue] Sorted color (ascending order): [Pink, Blue, Green, White, Black, Ivory, Orange, Indigo, Champagne]
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para ordenar un arreglo de cadenas por su longitud utilizando operaciones de la clase String (Manipulación de cadenas y estructuras de control).
Variables identificadas(tipo y uso)	Variables de tipo String, int y char según los argumentos requeridos por la lógica.
Operaciones , Formulas y Reglas :	Métodos aplicados: Manipulación de cadenas y estructuras de control. Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	Original unsorted colors: [Green, White, Black, Pink, Orange, Blue, Champagne, Indigo, Ivory] Sorted color (descending order): [Champagne, Orange, Indigo, Green, White, Black, Ivory, Pink, Blue] Sorted color (ascending order): [Pink, Blue, Green, White, Black, Ivory, Orange, Indigo, Champagne]
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia).

	4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos Manipulación de cadenas y estructuras de control e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 105: Tarea11_105 - Contar las Ocurrencias de una Subcadena en un Texto

Comprensión del Problema	
Descripción del problema:	Contar las Ocurrencias de una Subcadena en un Texto. Write a Java program to count the occurrences of a given string in another given string.
Entradas requeridas:	String str = "abcd abc aabc baa abcaa" String target = "aa" int count = 0
Salidas esperadas:	aa' has occured 3 times in 'abcd abc aabc baa abcaa'
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para contar las ocurrencias de una subcadena en un texto utilizando operaciones de la clase String (String.indexOf(), String.length()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"abcd abc aabc baa abcaa"' String target: Almacena la cadena de texto '"aa"' int count: Valor numérico o índice (0) int idx: Valor numérico o índice (0)
Operaciones , Formulas y Reglas :	Métodos aplicados: String.indexOf(), String.length(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	aa' has occured 3 times in 'abcd abc aabc baa abcaa'
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos.

	2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.indexOf(), String.length() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 106: Tarea11_106 - Concatenar una Cadena Consigo Misma N Veces

Comprensión del Problema	
Descripción del problema:	Concatenar una Cadena Consigo Misma N Veces. Write a Java program to concatenate a given string with itself a given number of times.
Entradas requeridas:	String str = "PHP" int times = 7
Salidas esperadas:	Original string: PHP After repeating 7 times: PHPPHPPHPPHPPHPPHPPHP
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para concatenar una cadena consigo misma n veces utilizando operaciones de la clase String (Manipulación de cadenas y estructuras de control).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"PHP"' int times: Valor numérico o índice (7)
Operaciones , Formulas y Reglas :	Métodos aplicados: Manipulación de cadenas y estructuras de control. Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	Original string: PHP After repeating 7 times: PHPPHPPHPPHPPHPPHPPHP
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	

¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos Manipulación de cadenas y estructuras de control e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 107: Tarea11_107 - Contar Ocurrencias de un Carácter Determinado

Comprensión del Problema	
Descripción del problema:	Contar Ocurrencias de un Carácter Determinado. Write a Java program to count occurrences of a certain character in a given string.
Entradas requeridas:	String str = "Red Green Blue Yellow" char ch = 'e' int count = 0
Salidas esperadas:	Resultado formateado en consola.
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para contar ocurrencias de un carácter determinado utilizando operaciones de la clase String (String.charAt(), String.length()).
Variables identificadas(tipo y uso)	String str: Almacena la cadena de texto '"Red Green Blue Yellow"' char ch: Carácter objetivo ('e') int count: Valor numérico o índice (0)
Operaciones , Formulas y Reglas :	Métodos aplicados: String.charAt(), String.length(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	Resultado formateado en consola.
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	

¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.charAt(), String.length() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 108: Tarea11_108 - Verificar si Existen Letras Consecutivas Idénticas

Comprensión del Problema	
Descripción del problema:	Verificar si Existen Letras Consecutivas Idénticas. Write a Java program to check whether there are two consecutive (following each other continuously), identical letters in a given string.
Entradas requeridas:	Cadenas o parámetros de texto definidos en el método principal (main).
Salidas esperadas:	Original word: Follow If there are two consecutive identical letters in the said string: true Original word: Java If there are two consecutive identical letters in the said string: false Original word: Yellow If there are two consecutive identical letters in the said string: true
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para verificar si existen letras consecutivas idénticas utilizando operaciones de la clase String (String.charAt(), String.length()).
Variables identificadas(tipo y uso)	Variables de tipo String, int y char según los argumentos requeridos por la lógica.
Operaciones , Formulas y Reglas :	Métodos aplicados: String.charAt(), String.length(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	Original word: Follow If there are two consecutive identical letters in the said string: true Original word: Java

	If there are two consecutive identical letters in the said string: false Original word: Yellow If there are two consecutive identical letters in the said string: true
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.charAt(), String.length() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 109: Tarea11_109 - Invertir Palabras cuya Longitud sea Impar

Comprensión del Problema	
Descripción del problema:	Invertir Palabras cuya Longitud sea Impar. Write a Java program that reverses all odd-length words in a string.
Entradas requeridas:	Cadenas o parámetros de texto definidos en el método principal (main).
Salidas esperadas:	Original text: Check two consecutive identical letters in a given string Reverses the words in the string that have odd lengths: Check two consecutive identical letters ni a given gnirts Original text: Create a Date object using the Calendar class Reverses the words in the string that have odd lengths: etaerC a etaD tcejbo using the radnelaC class
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para invertir palabras cuya longitud sea impar utilizando operaciones de la clase String (String.length(), String.split(), String.reverse()).

Variables identificadas(tipo y uso)	Variables de tipo String, int y char según los argumentos requeridos por la lógica.
Operaciones , Formulas y Reglas :	Métodos aplicados: String.length(), String.split(), String.reverse(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	Original text: Check two consecutive identical letters in a given string Reverses the words in the string that have odd lengths: Check two consecutive identical letters ni a given gnirts Original text: Create a Date object using the Calendar class Reverses the words in the string that have odd lengths: etaerC a etaD tcejbo using the radnelaC class
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.length(), String.split(), String.reverse() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 110: Tarea11_110 - Contar Caracteres Alfanuméricos que Ocurren Más de Dos Veces

Comprensión del Problema	
Descripción del problema:	Contar Caracteres Alfanuméricos que Ocurren Más de Dos Veces. Write a Java program to count the number of characters (alphanumeric only) that occur more than twice in a given string.
Entradas requeridas:	char lower = Character.toLowerCase(c) int count = 0
Salidas esperadas:	Original String: abcdaa Number of duplicate characters in the said String (Occurs more than twice.): 1 Original String: Create a Date Calendar. Number of duplicate characters in the said String (Occurs more than twice.):

	3
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para contar caracteres alfanuméricos que ocurren más de dos veces utilizando operaciones de la clase String (String.toCharArray(), String.toLowerCase()).
Variables identificadas(tipo y uso)	char lower: Carácter objetivo (Character.toLowerCase(c)) int count: Valor numérico o índice (0)
Operaciones , Formulas y Reglas :	Métodos aplicados: String.toCharArray(), String.toLowerCase(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	Original String: abcdaa Number of duplicate characters in the said String (Occurs more than twice.): 1 Original String: Create a Date Calendar. Number of duplicate characters in the said String (Occurs more than twice.): 3
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.toCharArray(), String.toLowerCase() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 111: Tarea11_111 - Eliminar una Palabra Específica de un Texto

Comprensión del Problema	
Descripción del problema:	Eliminar una Palabra Específica de un Texto. Write a Java program that

	removes a specified word from given text. Return the updated string..
Entradas requeridas:	String text = "Exercises Practice Solution" String word = "Solution"
Salidas esperadas:	Input a string: Exercises Practice Solution Input a word to remove: Solution New string, after removing the said word: Exercises Practice
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para eliminar una palabra específica de un texto utilizando operaciones de la clase String (String.replace(), String.trim()).
Variables identificadas(tipo y uso)	String text: Almacena la cadena de texto '"Exercises Practice Solution"' String word: Almacena la cadena de texto '"Solution"'
Operaciones , Formulas y Reglas :	Métodos aplicados: String.replace(), String.trim(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	Input a string: Exercises Practice Solution Input a word to remove: Solution New string, after removing the said word: Exercises Practice
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente. 3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.replace(), String.trim() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.

Ejercicio 112: Tarea11_112 - Comprobar si Todos los Caracteres de una Cadena Existen en Otra

Comprensión del Problema	
Descripción del problema:	Comprobar si Todos los Caracteres de una Cadena Existen en Otra. A string is created by using another string's letters. Letters are case sensitive. Write a Java program that checks the letters of the second string are present in the first string. Return true otherwise false.
Entradas requeridas:	String str1 = "Java" String str2 = "Ja"
Salidas esperadas:	Input the first string: Java Input the second string: Ja Check first string contains letters from the second string: true
Restricciones :	Cadenas no nulas (no null). Índices dentro del rango válido [0, longitud - 1]. Respetar mayúsculas y minúsculas.
Análisis	
¿Qué se necesita calcular o procesar?	Procesar la cadena de entrada para comprobar si todos los caracteres de una cadena existen en otra utilizando operaciones de la clase String (String.charAt(), String.indexOf(), String.length()).
Variables identificadas(tipo y uso)	String str1: Almacena la cadena de texto '"Java"' String str2: Almacena la cadena de texto '"Ja"'
Operaciones , Formulas y Reglas :	Métodos aplicados: String.charAt(), String.indexOf(), String.length(). Indexación basada en cero. Inmutabilidad de objetos String.
Salidas esperadas:	Input the first string: Java Input the second string: Ja Check first string contains letters from the second string: true
Restricciones :	Ninguna adicional; validación de límites para evitar StringIndexOutOfBoundsException.
Estrategia de Solución	
¿Qué pasos generales se seguirán?	1. Definir e inicializar las variables con las cadenas e índices requeridos. 2. Aplicar el método de String o algoritmo correspondiente.

	3. Validar las condiciones necesarias (comparación, rango o existencia). 4. Mostrar el resultado por consola.
¿Qué operaciones serán necesarias?	Invocación de métodos String.charAt(), String.indexOf(), String.length() e impresión mediante System.out.println().
¿Qué decisiones habrá que tomar?	Comprobar condiciones con if/else para verificar índices válidos, coincidencias de caracteres o subcadenas.
¿Qué datos intermedios se generarán?	Variables temporales numéricas (índices/contadores), subcadenas intermedias o booleanos de resultado.